

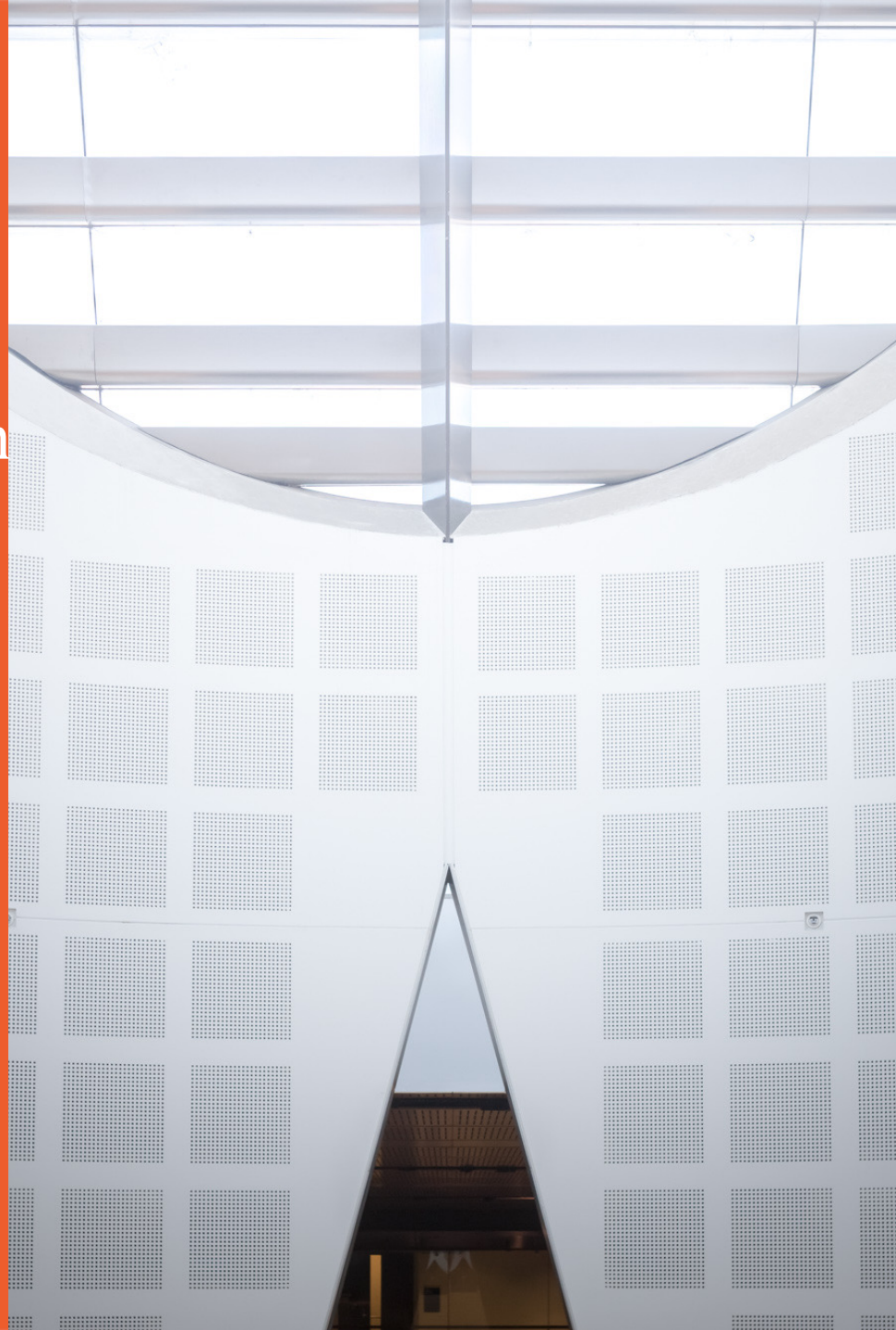
Self-Supervised Representation Learning

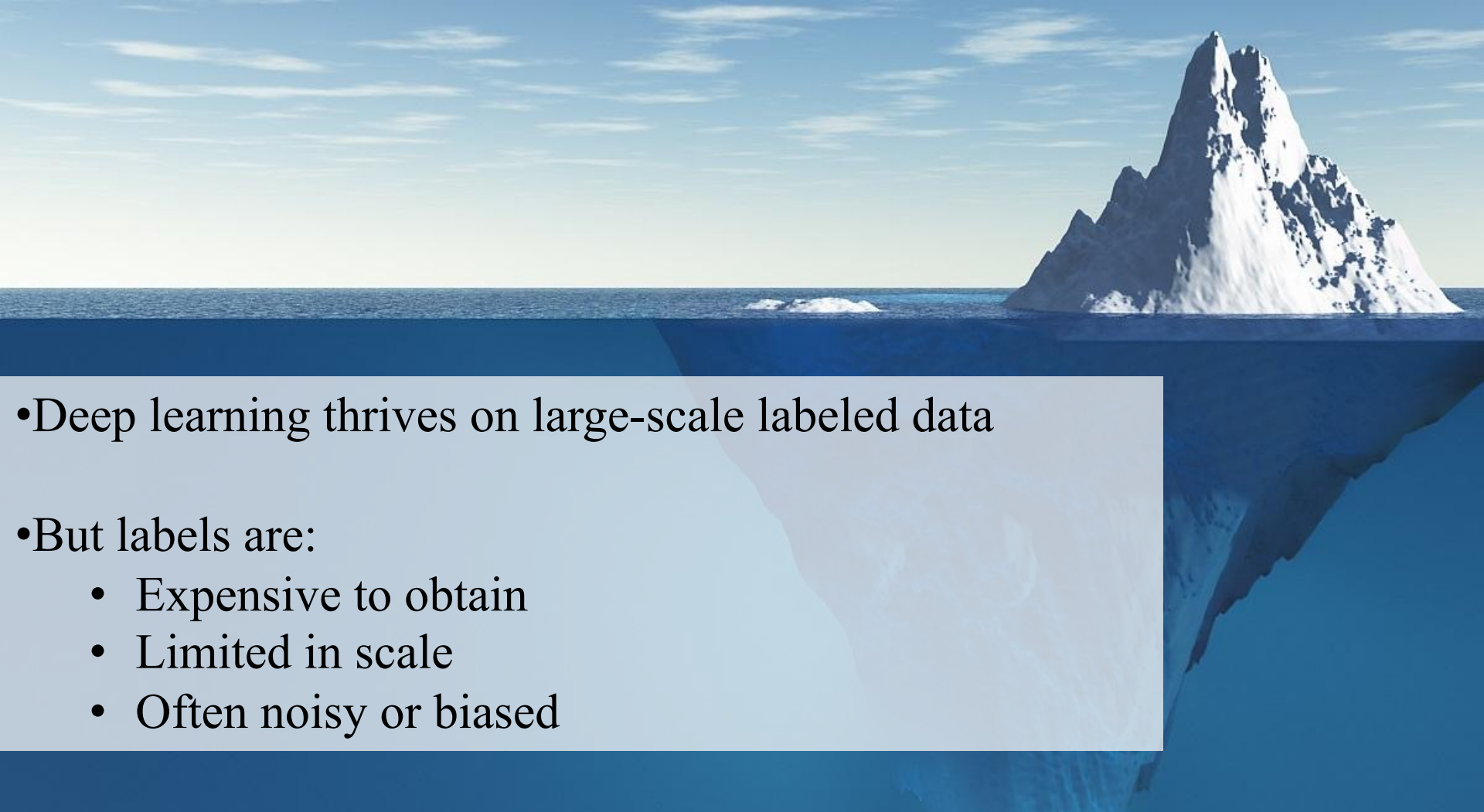
A/Prof Chang Xu

School of Computer Science



THE UNIVERSITY OF
SYDNEY





- Deep learning thrives on large-scale labeled data
- But labels are:
 - Expensive to obtain
 - Limited in scale
 - Often noisy or biased

Models are limited by labels, not by data.

Are We Fully Using the Data?

- Supervised learning uses only label information
- Labels provide extremely limited signals
 - e.g., one-hot classification targets
- But data contains rich structure:
 - Spatial patterns
 - Object relationships
 - Context and semantics



Learning Without Labels (Human Perspective)

Humans do not rely on explicit labels

Learning is driven by:

- Observation
- Prediction
- Completion
- Comparison



Self-Supervised Representation Learning

Learn representations without manual labels

Supervision is derived from data itself

Design pretext tasks to create learning signals

Self-Supervised Representation Learning

Generative SSL

- Predict missing or corrupted content.
- Loss is reconstruction in pixel/patch/token space.
- Examples: Autoencoder, MAE.
- Strength: forces feature retention.
- Risk: low-level shortcuts dominate.

Discriminative SSL

- Match representations, not pixels.
- Loss is similarity/classification in embedding space.
- Contrastive: SimCLR, MoCo (need negatives).
- Non-contrastive: BYOL, SimSiam, DINO (no negatives).
- Multimodal: CLIP (cross-modal contrastive).

Generative Self-Supervision

AutoEncoder

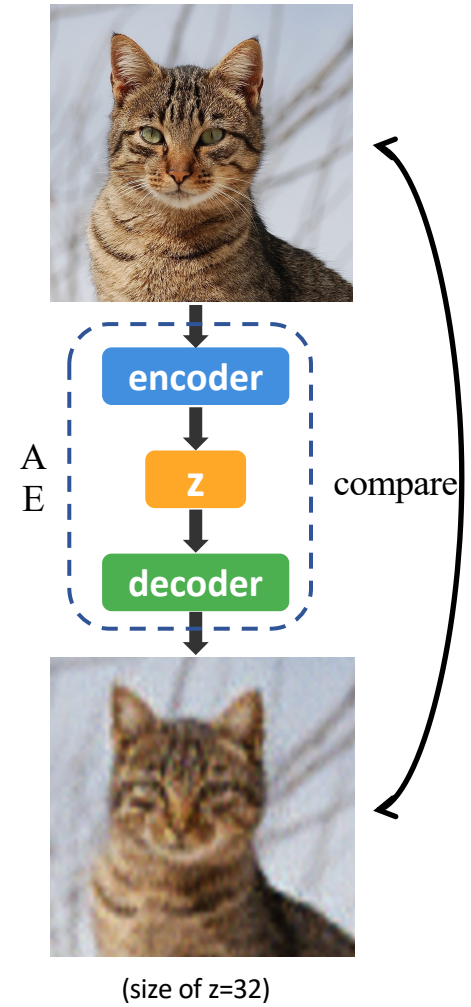
Early idea in **unsupervised representation learning**.

Learn a representation by **reconstructing the input** from itself.

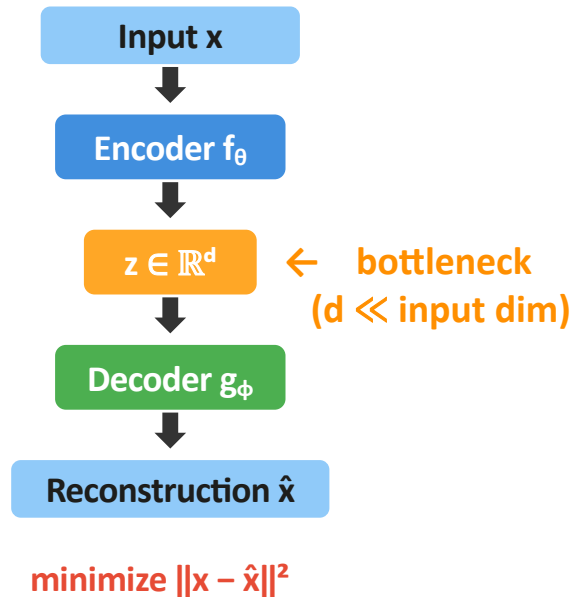
Compression forces the model to keep useful structure.

What it learns: **features** that preserve information.

What it does **NOT** learn: **invariance** or **discriminativeness**.



Autoencoder: architecture



Encoder $\mathbf{f}_\theta$ **compresses** input to **low-dim** latent code $\mathbf{z}$.

Decoder $\mathbf{g}_\phi$ **reconstructs** original input **from $\mathbf{z}$** alone.

The **bottleneck $\mathbf{z}$** is the key of the method:

- d too large $\rightarrow$ identity copy (no useful learning)
- d too small $\rightarrow$ loses important structure

Latent code $\mathbf{z}$ is the learned representation.

- After training, decoder is optional for downstream tasks.

Autoencoder: objective function

$$L_{AE} = \frac{1}{N} \sum_i \|x_i - g_\varphi(f_\theta(x_i))\|^2$$

$$\mathbf{z}_i = f_\theta(\mathbf{x}_i) \in \mathbb{R}^d, \quad \hat{\mathbf{x}}_i = g_\varphi(\mathbf{z}_i) \in \mathbb{R}^D, \quad d \ll D$$

$$\|\mathbf{x}_i - g_\varphi(f_\theta(\mathbf{x}_i))\|^2 = \sum_{j=1}^D (x_i^{(j)} - \hat{x}_i^{(j)})^2$$

$$L_{AE} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^D (x_i^{(j)} - \hat{x}_i^{(j)})^2$$

Pixel level reconstruction loss

Undercomplete (d < input dim):

Forces compression. Linear case = PCA.

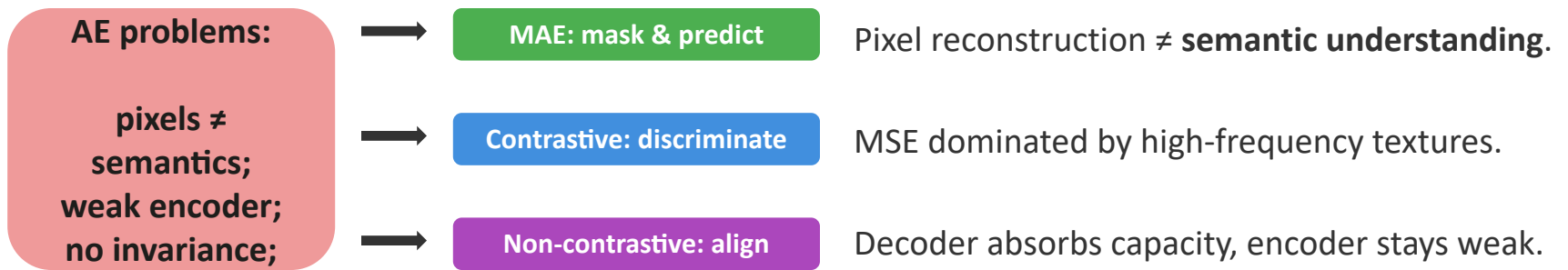
Overcomplete (d >= input dim):

Can learn identity. Need regularization.

Denoising AE (Vincent 2008):

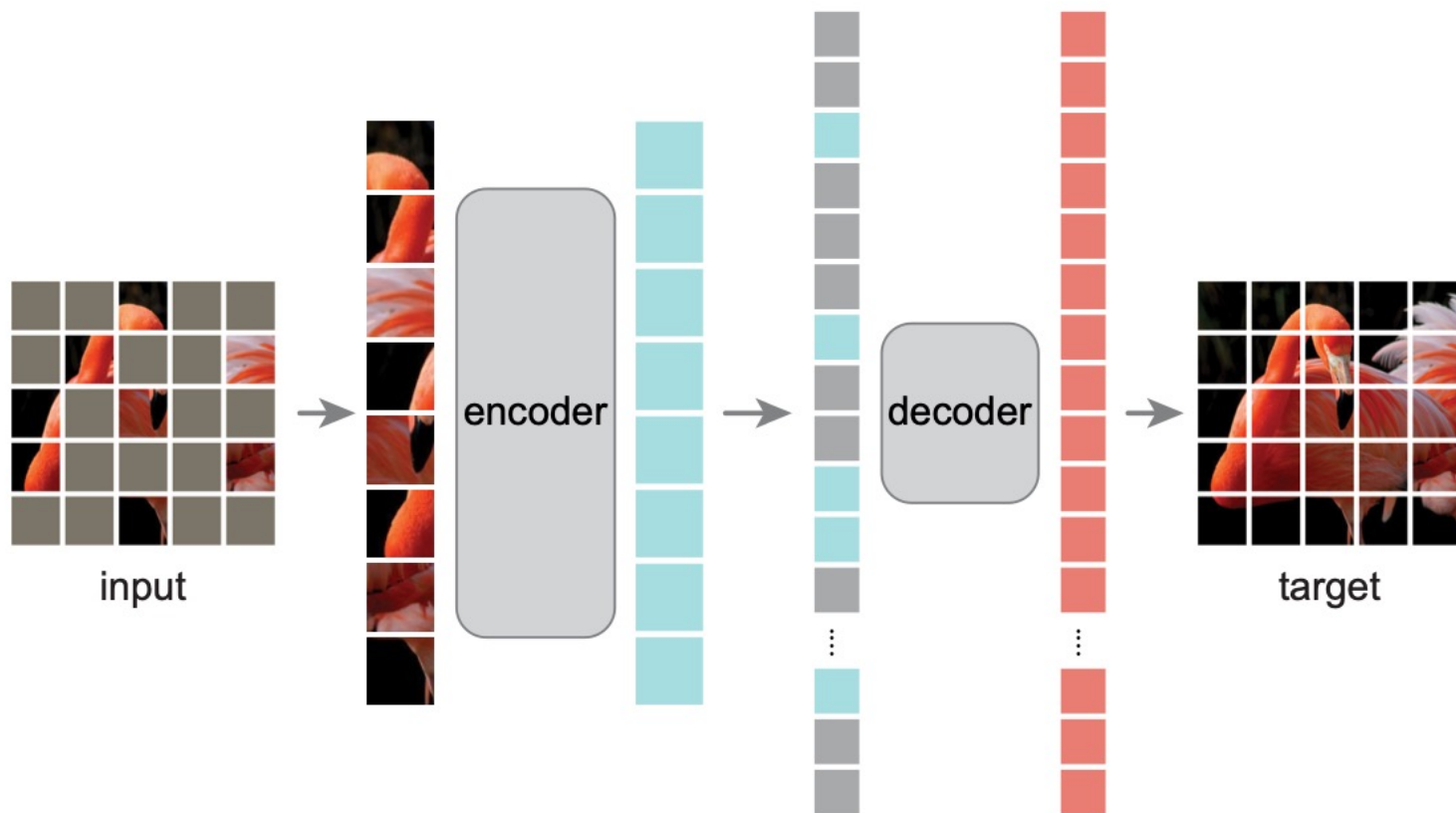
Corrupt input, reconstruct clean. Robust features.

Autoencoder limitations: why the field moved on

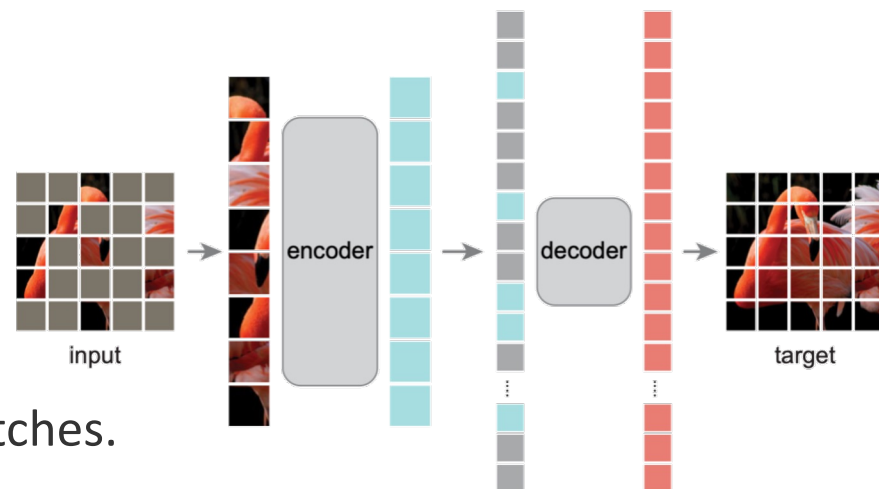


Each successor fixes a specific limitation.

Masked Autoencoders (MAE)



MAE: architecture



Patchify: split image into non-overlapping patches.

Random mask: sample 75% of patches to hide.

Encoder: ViT applied only to the 25% visible patches.
(Mask tokens are **NOT** fed to the encoder.)

Decoder input: visible tokens + learnable mask tokens.

Decoder: lightweight ViT (8 blocks) reconstructs pixels.

Loss computed **only on masked patch** positions.

MAE: objective function

$$L_{MAE} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{\mathbf{x}}_i - \mathbf{x}_i\|^2$$

$\mathcal{M} \subset \{1, 2, \dots, N\}$: masked patch indices (random 75%)

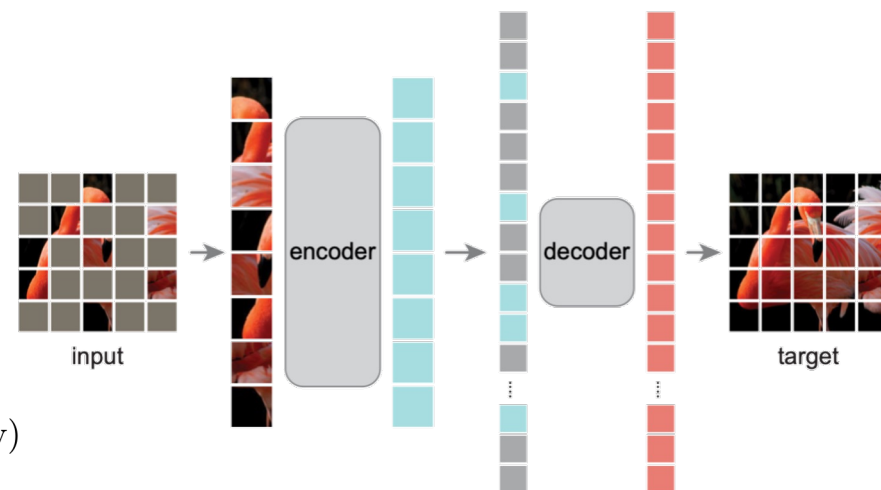
$\mathcal{V} = \{1, \dots, N\} - \mathcal{M}$: visible patch indices (25%)

Encoder only sees visible patches $\{\mathbf{x}_j\}_{j \in \mathcal{V}}$, NOT all $\mathbf{x}_i$

Forward from input image

$$\mathbf{h}_i = f_{\theta}(\{\mathbf{x}_j\}_{j \in \mathcal{V}})_i, \quad i \in \mathcal{V} \cup \mathcal{M}$$

$$\hat{\mathbf{x}}_i = g_{\varphi}(\mathbf{h}_i), \quad i \in \mathcal{M} \quad (\text{reconstruct masked patches only})$$



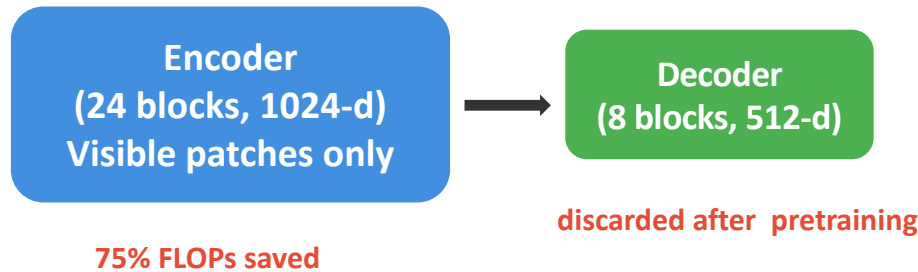
Backward from loss

Loss **only** on masked positions.

$$\frac{\partial L}{\partial \varphi} = \frac{2}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} (\hat{\mathbf{x}}_i - \mathbf{x}_i) \cdot \frac{\partial g_{\varphi}}{\partial \varphi}$$

$$\frac{\partial L}{\partial \theta} = \frac{2}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} (\hat{\mathbf{x}}_i - \mathbf{x}_i) \cdot \frac{\partial g_{\varphi}}{\partial \mathbf{h}_i} \cdot \frac{\partial f_{\theta}}{\partial \theta}$$

MAE: why the asymmetric design works



Encoder: heavy, visible patches only

ViT-L: 24 blocks, 1024-d. Processes only 25% of patches.

Mask tokens NOT in encoder -- this is the key trick.

Decoder: lightweight, pretraining only

8 blocks, 512-d. Sees all tokens (visible + mask).

Discarded for downstream. Keeps encoder capacity focused.

Downstream: encoder only

Fine-tune or linear probe. Representation = encoder output.

MAE: why 75% masking is surprisingly effective

Too little masking (e.g., 25%):

Task is too easy -- local texture interpolation suffices.
Encoder does not need to understand global structure.

case	ratio	ft	lin
random	75	84.9	73.5
block	50	83.9	72.3
block	75	82.8	63.9
grid	75	84.0	66.0

Too much masking (e.g., 95%):

Not enough visible context to reason about the image.
Diminishing returns on representation quality.



Sweet spot: 75% random masking

Forces encoder to integrate global structure from sparse evidence.

Random sampling (not block/grid) works best -- see table.

Compare with NLP: BERT masks only 15%.

Images have far more spatial redundancy -- need heavier masking.

MAE vs classic AutoEncoder

AE Encoder input : all patches $\{\mathbf{x}_i\}_{i=1}^N$ vs MAE Encoder input : visible only $\{\mathbf{x}_j\}_{j \in \mathcal{V}}$

AE Loss : reconstruct all N patches vs MAE Loss : reconstruct only $|\mathcal{M}|$ masked patches

Classic AE

- Reconstruct entire input
- Easy task
- Low semantic pressure
- Encoder sees everything
- Symmetric encoder-decoder

MAE

- Reconstruct only masked 75%
- Hard task -- must infer missing
- High semantic pressure
- Encoder sees 25% only
- Asymmetric (heavy enc, light dec)

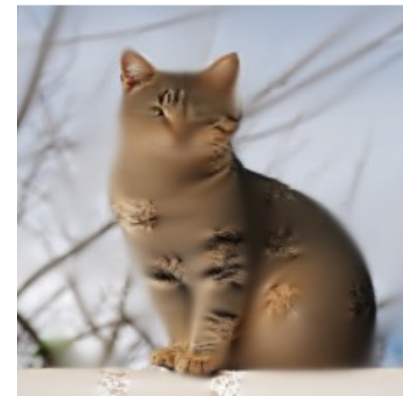
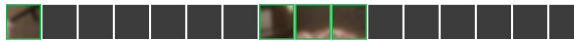
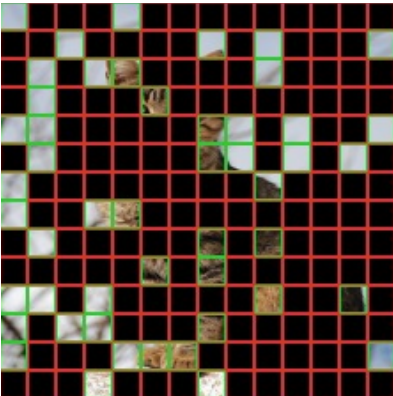
MAE is NOT 'AE with masking'! They are fundamentally different pretext task.

MAE vs AE in forwarding

AE



MAE

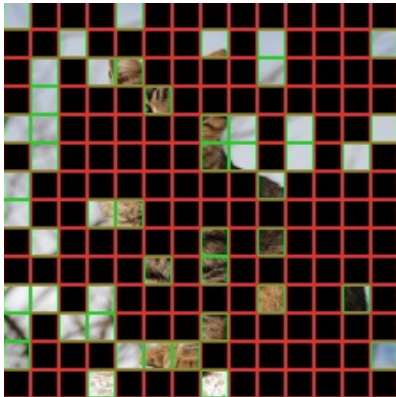


Input x

Latent Space z

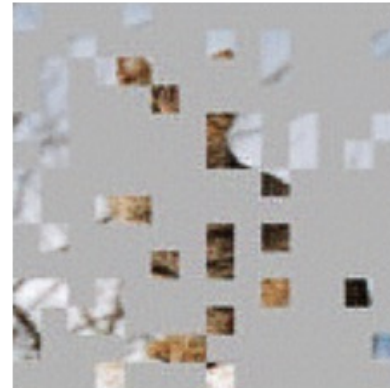
Output $\hat{x}$

MAE vs AE on masked image reconstruction

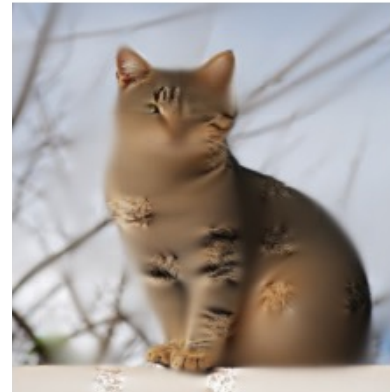


Shared Input x

AE



MAE



Output $\hat{x}$

Contrastive Learning

From generative to contrastive learning

Basic Image for Contrastive:

No reconstruction. No decoder needed.

Positive pairs: two views of same image.

Negatives pairs: views of other images.

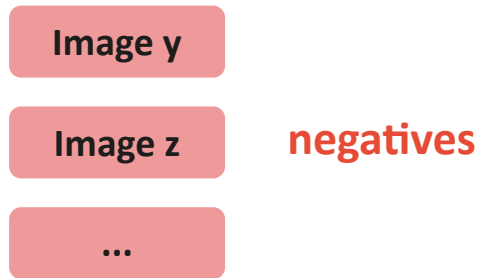
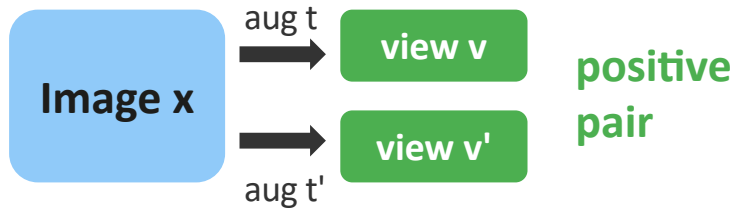
Pull same together, **push different** apart.

Augmentation defines the invariances.

Generative:
reconstruct pixels

Contrastive:
discriminate instances

Positive and negative pairs



Positive pair:

Same image, different random augmentations (crop, color, blur, flip).

The model learns: these two views should have **similar representations**.

Negatives pair:

Views from different samples in the same batch.

The model learns: these samples should have **different representations**.

Some example transforms

The ColorJitter transform randomly changes the brightness, saturation, and other properties of an image.

Original image

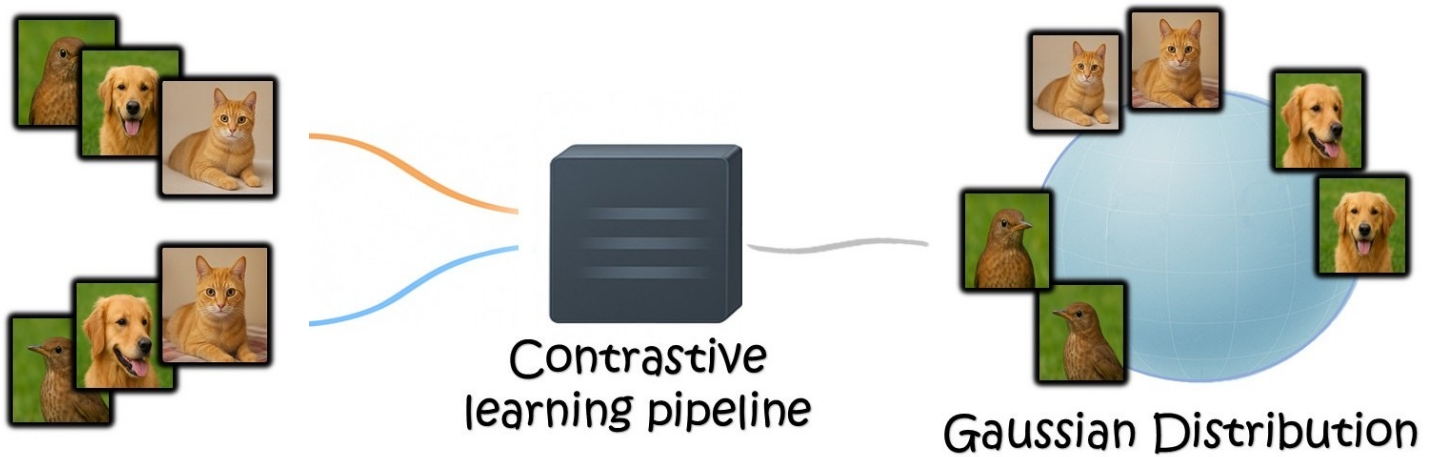


The GaussianBlur transform performs gaussian blur transform on an image.

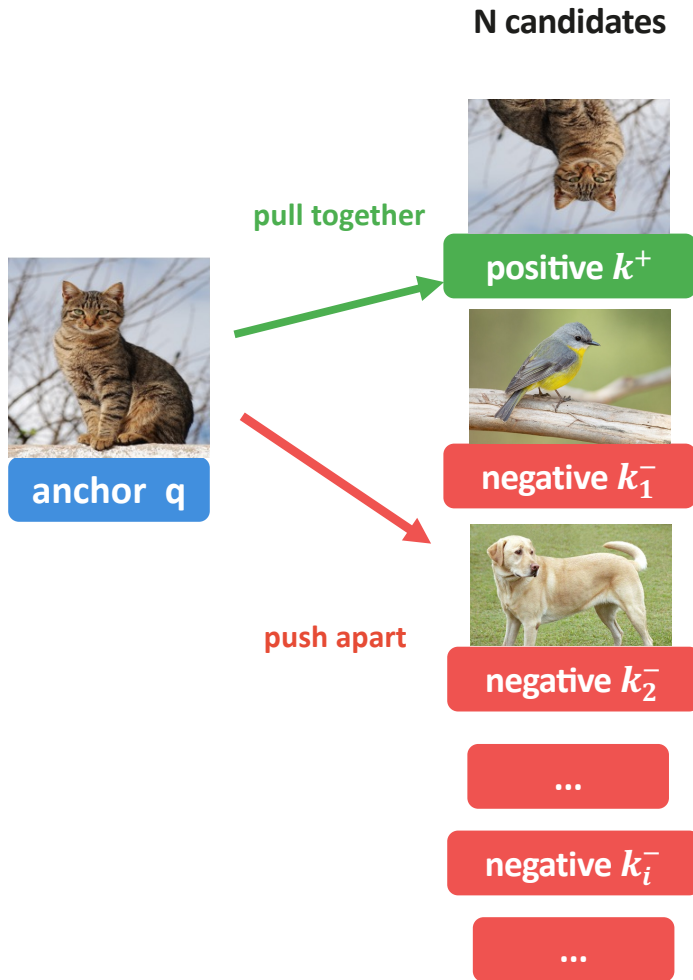
Original image



InfoNCE



InfoNCE: intuition



Think of it as an N-way classification problem:

Given an anchor, identify which of N candidates is the positive. The other N-1 are negatives.

The model outputs a similarity score for each. Apply softmax over scores.

Minimize cross-entropy with the true positive.

This is 'instance discrimination':

Each image is its own class.

Two augmented views share the same 'class'.

All other images are different 'classes'.

More negatives = harder task = better features.

InfoNCE: the objective function

$$L = -\log \frac{\exp(q \cdot k^+ / \tau)}{\exp(q \cdot k^+ / \tau) + \sum_i \exp(q \cdot k_i^- / \tau)}$$

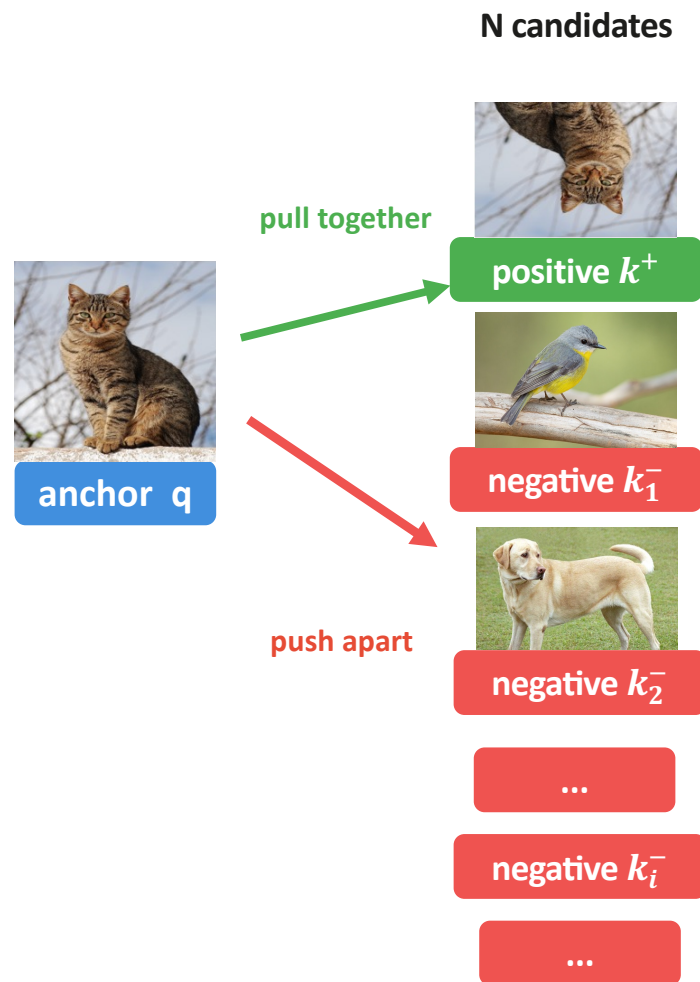
$\cos(q, k^+)$

softmax

Temperature τ : controls sharpness

Softmax over positive + negative

This is a classification problem: identify the positive among many candidates.

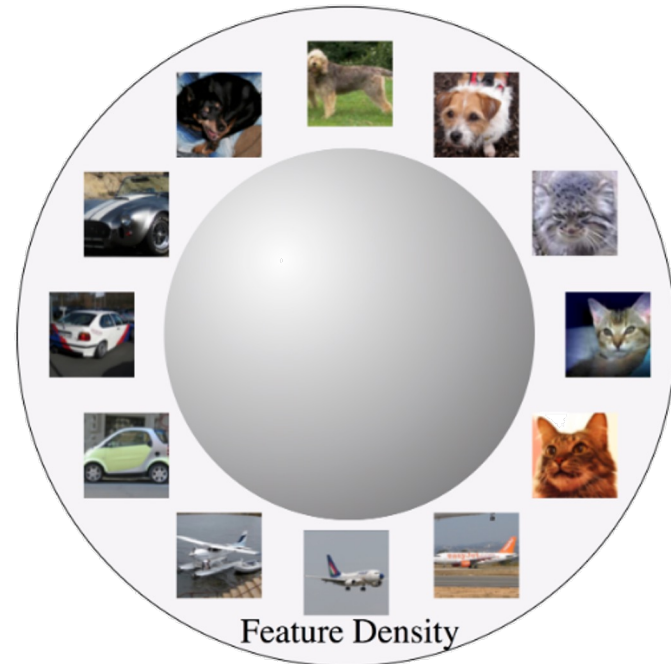
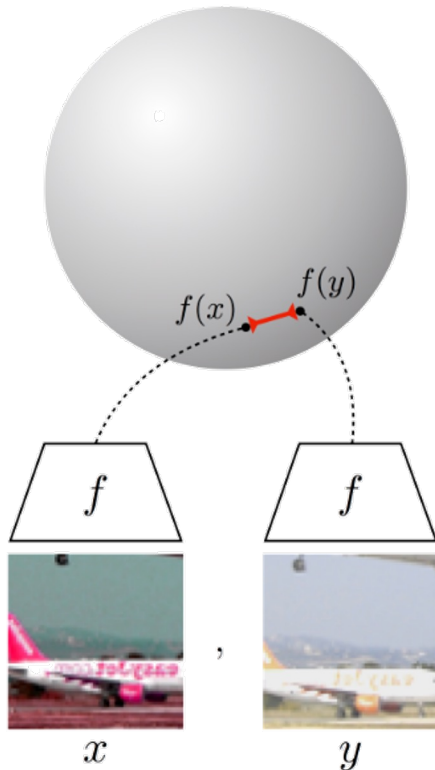


Geometric Interpretation

Pull positive pairs together

Push negatives apart

Normalize representations (often)



Gradients of InfoNCE

$$L = -\log \frac{\exp(q \cdot k^+ / \tau)}{\exp(q \cdot k^+ / \tau) + \sum_i \exp(q \cdot k_i^- / \tau)}$$

$$\frac{\partial L}{\partial q} = \frac{1}{\tau} \left(\sum_j p_j k_j - k^+ \right) \quad p_j = \frac{\exp(q \cdot k_j / \tau)}{\sum_i \exp(q \cdot k_i / \tau)}$$

gradient descent: $q \leftarrow q - \eta \frac{\partial L}{\partial q}$

The gradient has two forces:

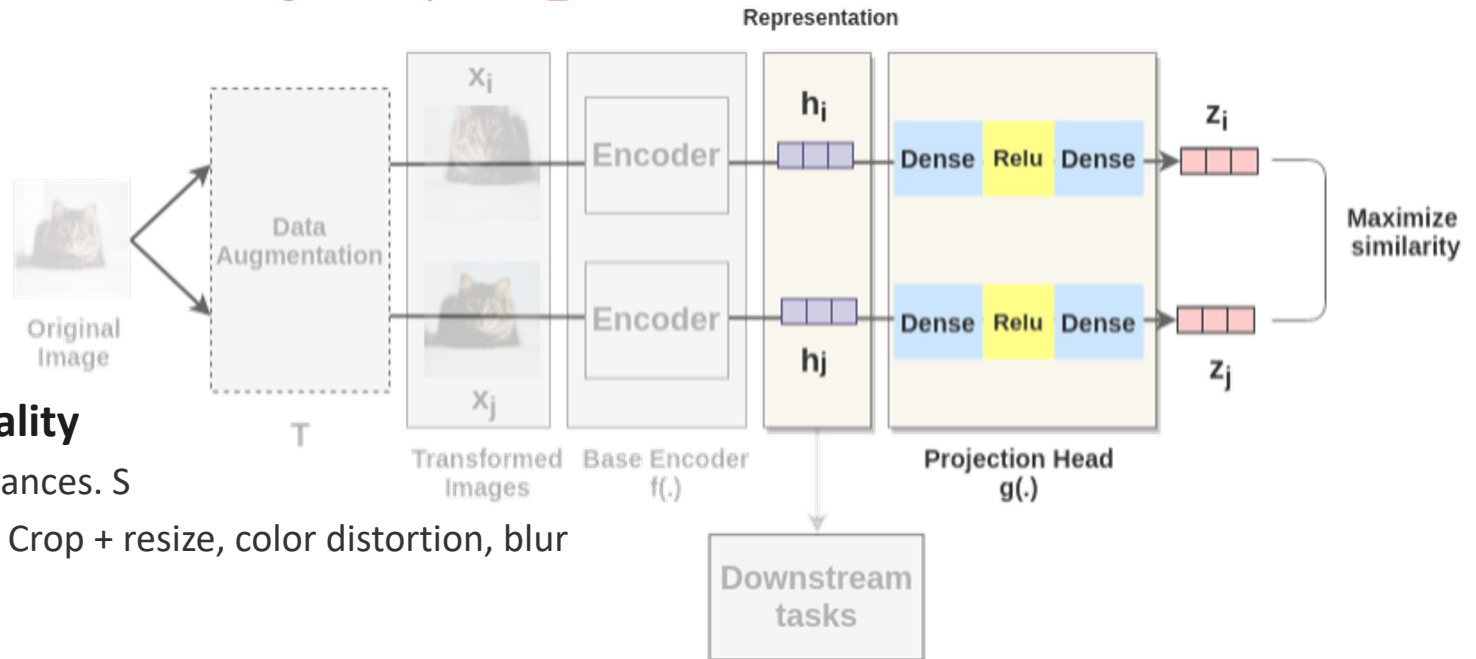
- A **pull force** toward the positive sample
- A **push force** away from a weighted average of negatives

$-k^+$

$\sum_j p_j k_j$

Not all negatives are equal — the model focuses on the **hard negatives**.

Contrastive learning: key dependencies/SimCLR



Augmentation quality

Defines learned invariances. S

Strong augmentation: Crop + resize, color distortion, blur

Enough negatives (e.g., 4096)

More negatives $\rightarrow$ better approximation of the data distribution

A larger set of negatives leads to stronger discrimination.

Temperature τ : controls how aggressively we separate points on the sphere.

small τ : the softmax becomes sharp; the hardest negatives dominate.

large τ : the distribution is smoother; all examples contribute.

Encoder & Projection head

Loss in z -space; downstream uses h -space.

SimCLR

Strengths

Simple, end-to-end, strong baseline.

Clean ablation platform.

69.3% linear probe (R50, batch 4096).

No memory bank, no momentum encoder.

SimCLR = pure batch-based contrastive learning.

What actually matters:

1. Augmentation composition: defines what invariances are learned.
2. Projection head: separates contrastive space from feature space.
3. Scale (batch size + epochs): more negatives samples = better representation learned.

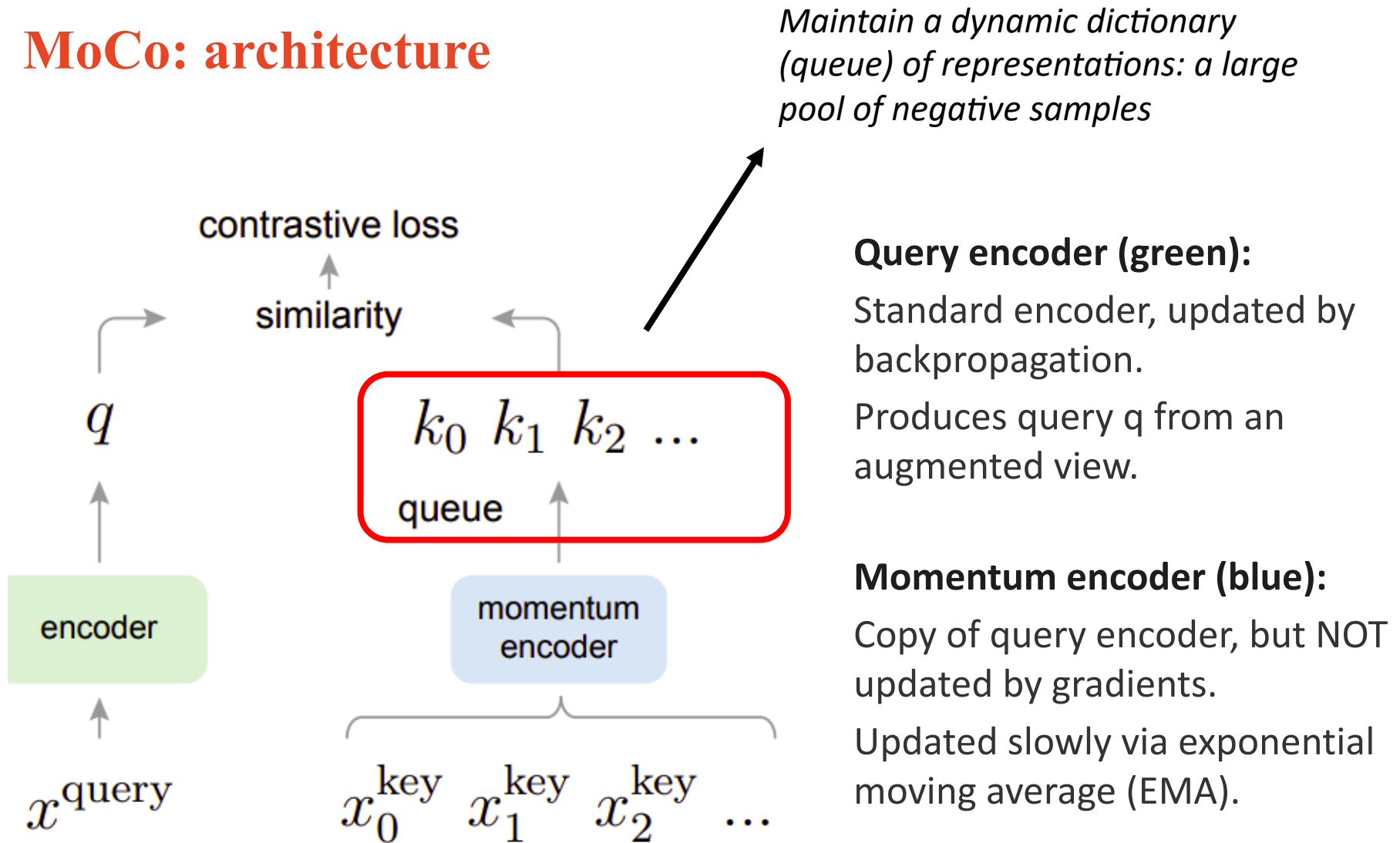
Limitations

Very dependent on **large batch size**.

Complexity: $O(N^2)$ for the all-pairs similarity matrix.

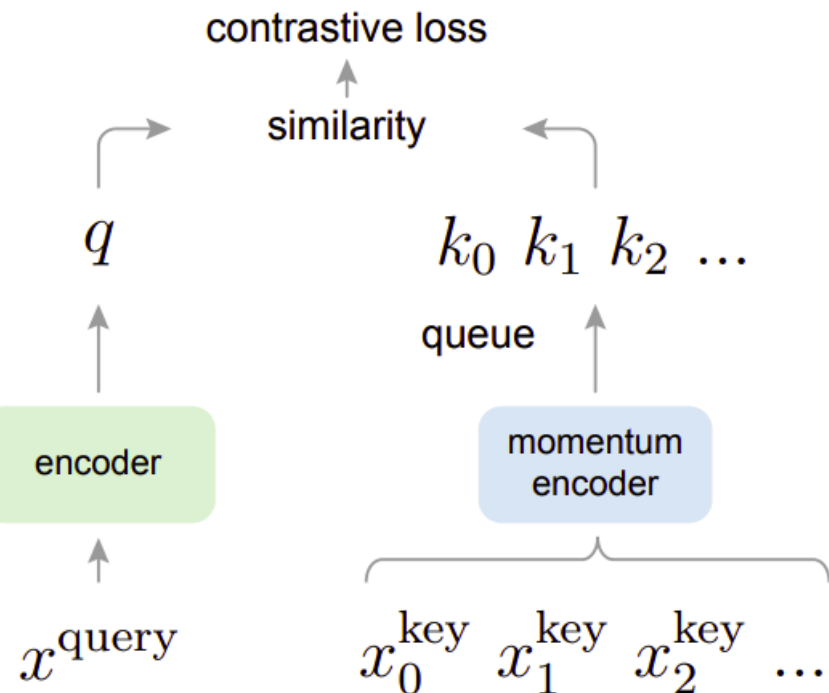
Next: MoCo asks -- can we keep many negatives without giant batches?

MoCo: architecture



MoCo replaces “large batch” with a “memory queue”.

MoCo: architecture



Queue:

FIFO buffer storing keys from previous minibatches.

Size $K = 65536$ (independent of batch size).

Each step: enqueue new keys, dequeue oldest.

Contrastive loss: Same as InfoNCE.

q should **match** its **positive key**, and be **dissimilar** to all **other keys** in the queue.

exponential moving average (EMA):

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$

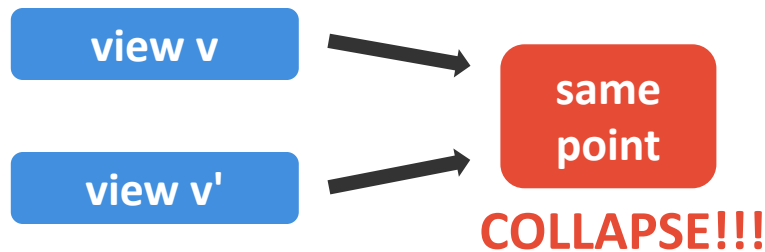
With $m=0.999$: 0.1% change per step. Queue stays consistent.

Momentum update ensures consistency of representations in the queue.

Non-Contrastive Learning

What if we remove all negative samples?

Without negatives: why does the problem get harder?



Representation collapse:

All inputs produce identical output regardless of content.

Training **loss** looks **great**. Downstream **accuracy** is **random**.

This is the central challenge of non-contrastive methods.

In contrastive learning:

Negatives provide **repulsion**: prevent all embeddings from **collapsing**.

The loss has two forces: attraction (positives) and repulsion (negatives).

Remove negatives:

Only attraction remains: $L = -\text{sim}(z, z')$

The trivial solution: map **EVERYTHING** to the **same** constant vector.

$\text{sim}(c, c) = 1$ for any constant c . Loss is minimized. Features are useless.

Non-contrastive: how each avoids collapse

Without negatives, how to avoid collapse?

BYOL

EMA target +
predictor

Stop-gradient on
target

SimSiam

Shared weights +
predictor

Stop-gradient only

DINO

EMA + centering +
sharpening

Multi-crop + cross-
entropy

Common: structural asymmetry between branches.

The unified view: asymmetry prevents collapse

All non-contrastive methods: align views of same image.

No negatives. No reconstruction target.

Why no collapse -- the asymmetry principle:

1. Asymmetric architecture:

Predictor on one branch only (BYOL, SimSiam).

2. Asymmetric optimization:

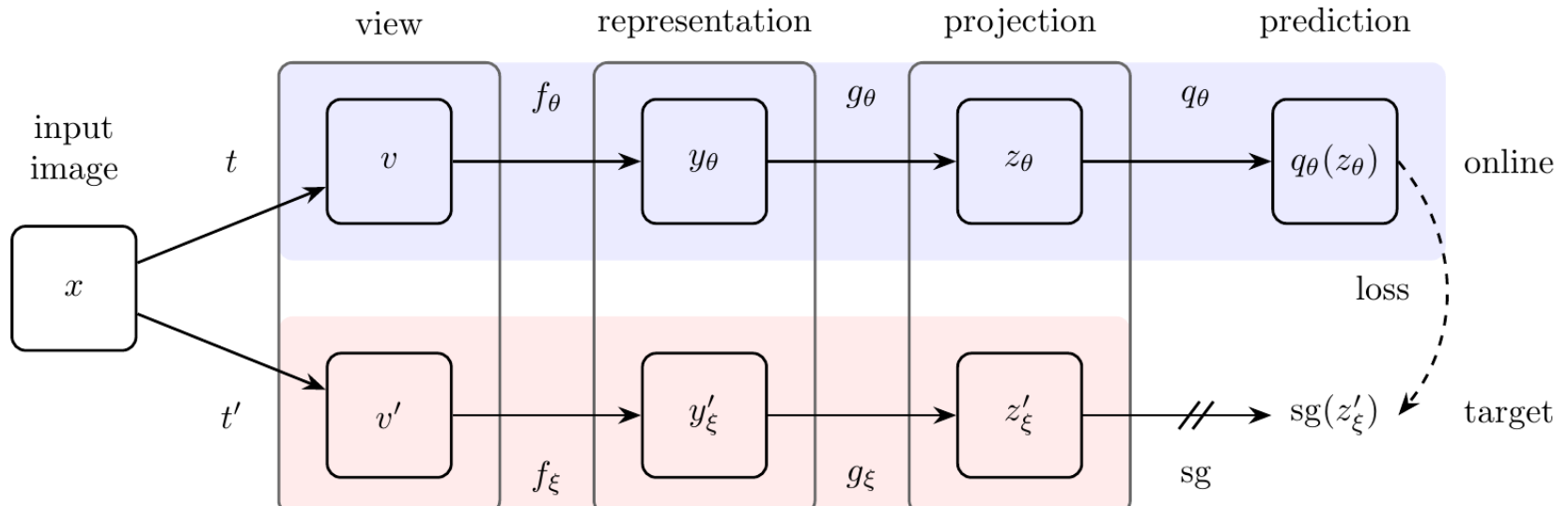
Stop-gradient + EMA update (BYOL, DINO).

3. Asymmetric targets:

Centering + sharpening + multi-crop (DINO).

'Asymmetry' is the single concept that unifies this section.

BYOL (Bootstrap Your Own Latent)



The target network evolves slowly: $\xi \leftarrow m\xi + (1 - m)\xi$

Online (blue):

encoder f_θ + projector g_θ + predictor q_θ

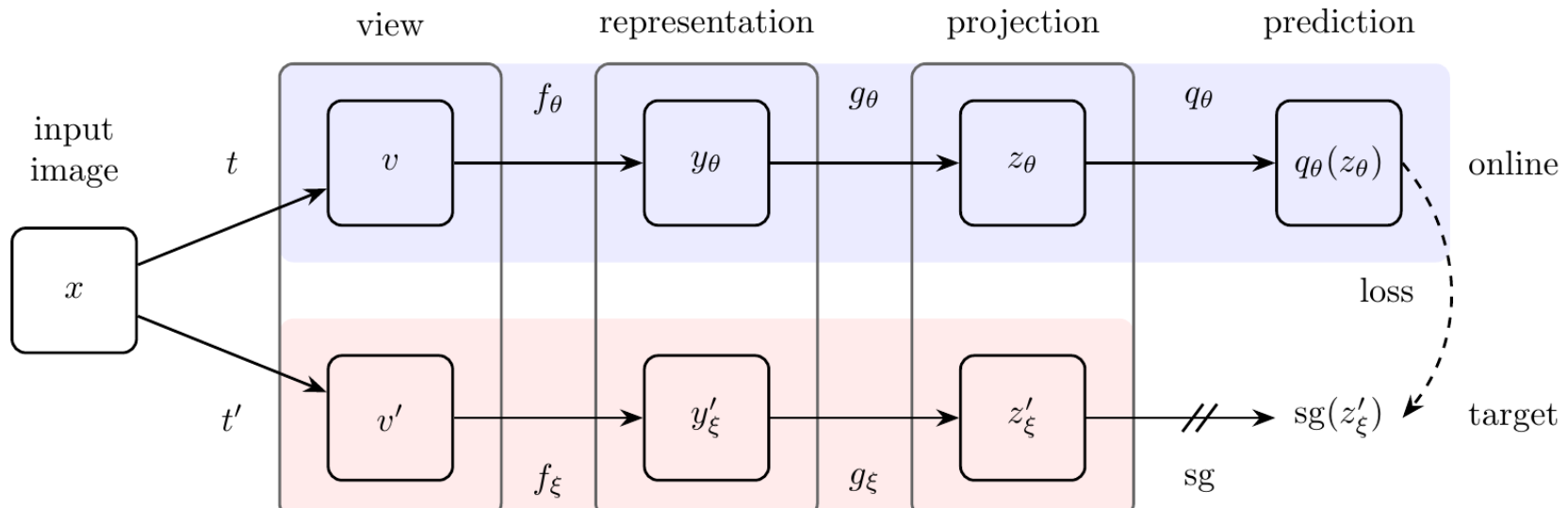
$$\bar{q}_\theta = \frac{q_\theta(z_\theta)}{\|q_\theta(z_\theta)\|_2}, \quad \bar{z}'_\xi = \frac{z'_\xi}{\|z'_\xi\|_2}$$

Target (pink):

encoder f_ξ + projector g_ξ (**NO predictor**)

$$\mathcal{L} = \|\bar{q}_\theta - \bar{z}'_\xi\|^2$$

Why does BYOL not collapse?



No single clean theorem. Primarily empirical.

1. Predictor = architectural asymmetry

Online has predictor; target does not. Most critical.

2. EMA = slowly moving target

Target lags behind. Cannot match fixed point.

3. Stop-gradient blocks co-adaptation

Target gets no gradients. Cannot cooperate to collapse.

BYOL learns by predicting a slowly evolving version of itself.

BYOL: key results

ImageNet linear eval (ResNet-50):

BYOL: 74.3%

SimCLR: 69.3% | MoCo v2: 71.1%

BYOL beats both -- without any negatives.

Larger models:

ResNet-200 (2x): 79.6%

Approaching supervised baselines (76.5% R50 supervised).

Transfer tasks:

Competitive on VOC detection, COCO, Places205.

Robust to augmentation variations.

The conceptual result matters more than the numbers:

Non-collapse is possible without negatives.

This opened the door to SimSiam and DINO.

BYOL: implementation checklist

EMA coefficient:

tau from 0.996 to 1.0 over training (cosine schedule).

Too low (0.9): target changes too fast, unstable.

Too high (1.0 from start): target is frozen, never learns.

Predictor -- do NOT remove it:

2-layer MLP, 4096 hidden, 256 output.

Without the predictor, BYOL collapses immediately.

This is the single most important component for avoiding collapse.

Augmentations still matter:

Same as SimCLR: random crop + color jitter + blur + solarize.

Even without negatives, augmentation defines the invariance learned.

Larger models help:

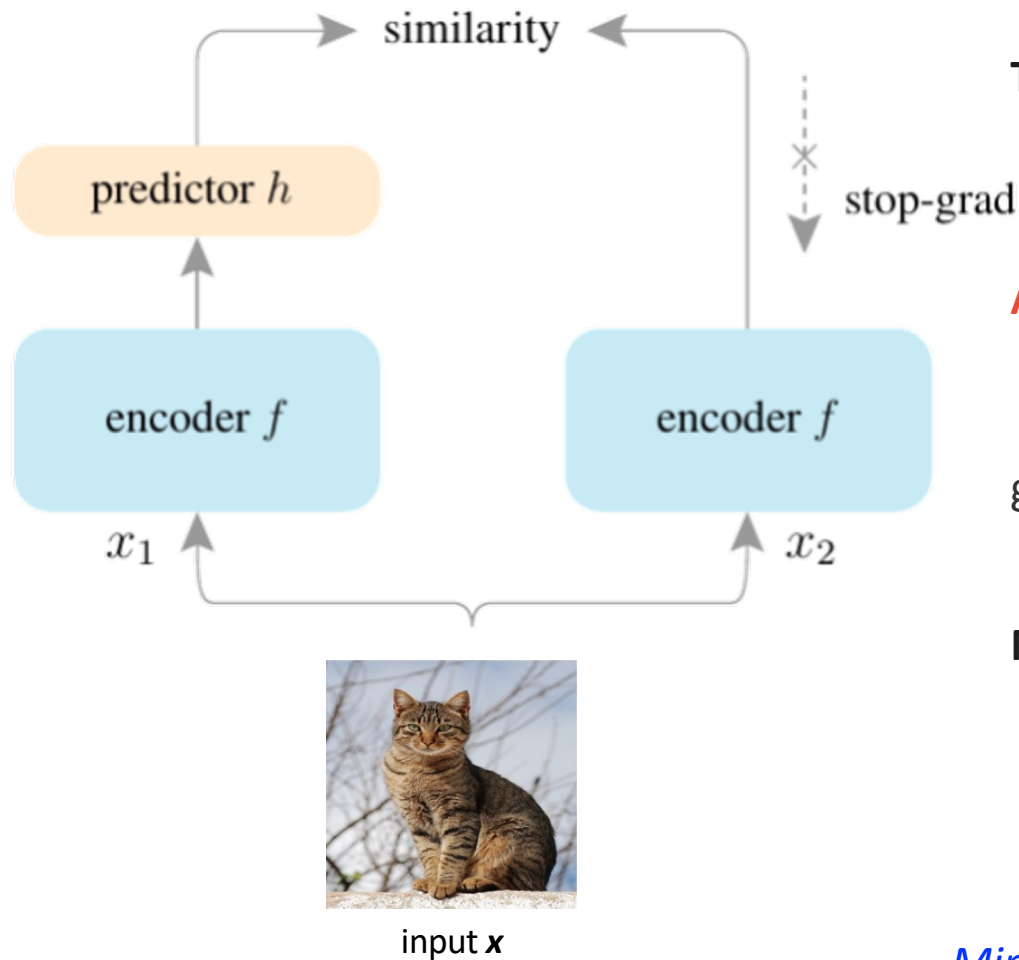
BYOL benefits more from model scaling than contrastive methods.

Optimizer: SGD + LARS, cosine LR, batch 4096, 1000 epochs.

Do We Need a Momentum Encoder?

BYOL removes negative samples, but still uses momentum target network

SimSiam



Two branches, same encoder:

Both use identical f (shared weights).

Asymmetry:

Branch 1: encoder + projector + predictor h .

Branch 2: encoder + projector + stop-gradient.

No EMA. No separate target.

Target = other branch output, frozen by sg.

Minimal recipe: predictor + stop-gradient.

SimSiam: objective function

$$\mathcal{L} = -\frac{1}{2} [\text{sim}(p_1, \text{sg}(z_2)) + \text{sim}(p_2, \text{sg}(z_1))]$$

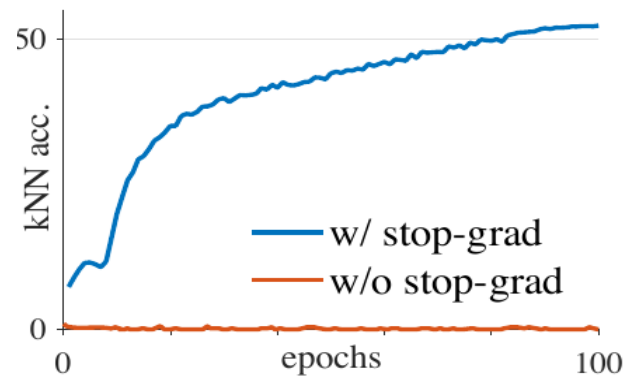
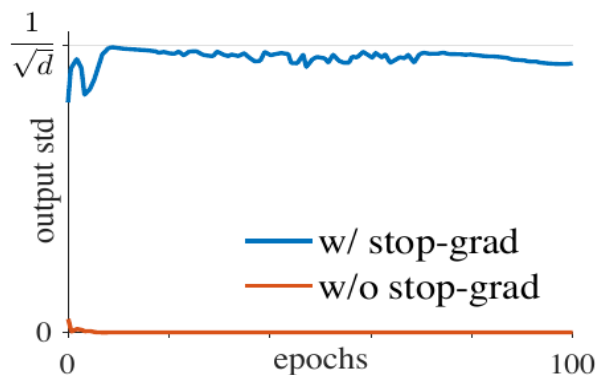
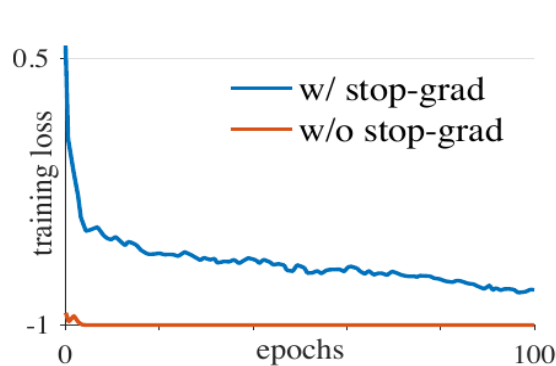
p=predictor out.

z=encoder out.

sg=stop-gradient.

Cosine similarity, symmetrized.

SimSiam: what stop-gradient actually does



Without sg: collapse (red, flat at -1).

Both branches converge to constant.

With sg: learning works (blue).

Alternating optimization: fix one, update other.

Also need predictor.

Remove predictor (even with sg) = collapse.

BYOL vs SimSiam: what is essential?

	BYOL	SimSiam
Target	EMA copy	shared weights
EMA	yes (τ 0.996 $\rightarrow$ 1.0)	none
Predictor	yes (essential)	yes (essential)
Stop-grad	on target	on target
Complexity	higher	minimal
IN-1K LP	74.3%	71.3%

Minimal recipe:

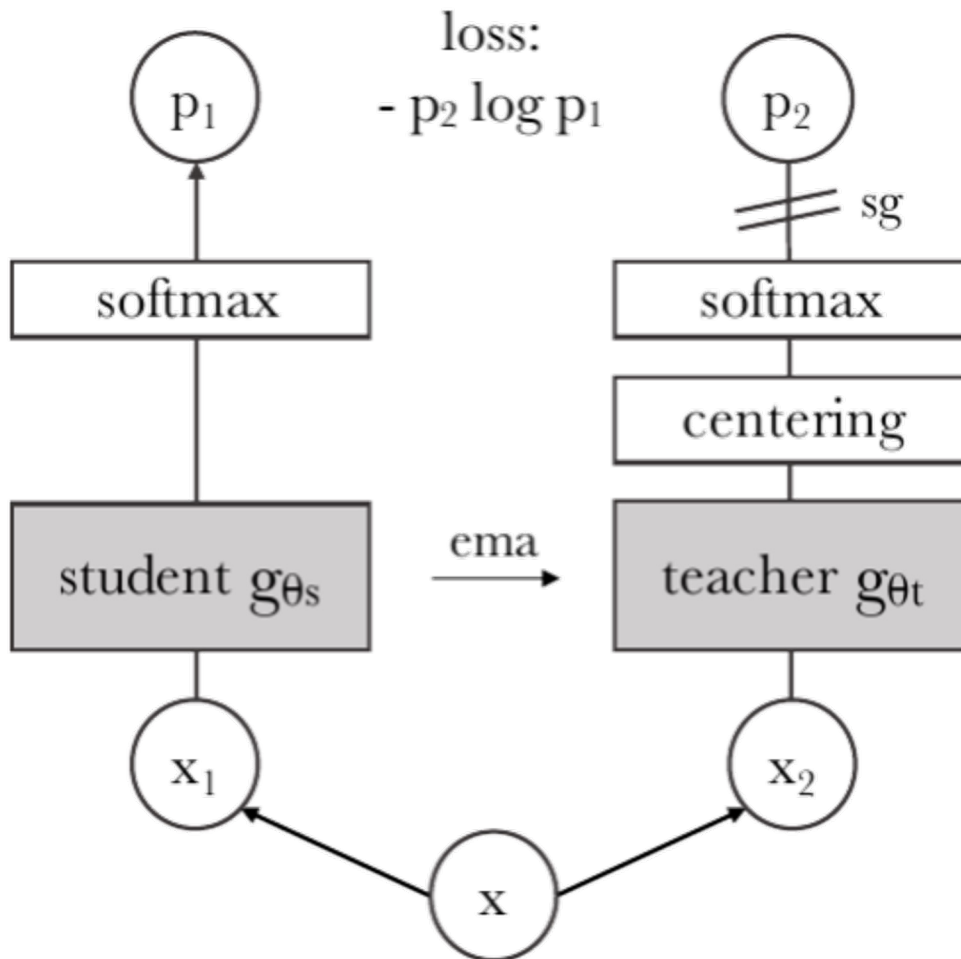
shared encoder + predictor + stop-gradient. EMA helps but is not essential.

BYOL / SimSiam: match representations (vector-level alignment)

Can we learn richer structure beyond simple matching?

DINO moves from vector matching to distribution matching.

DINO: Self-Distillation Without Labels



It uses softmax because it wants a structured space where distribution-level constraints can be imposed.

$$\tilde{z}_t = z_t - c, \quad c: \text{running mean}$$
$$c \leftarrow mc + (1 - m)\text{mean}(z_t)$$

The teacher–student mechanism in DINO is essentially the same as the *momentum* encoder in BYOL.

DINO: Self-Distillation Without Labels

After softmax

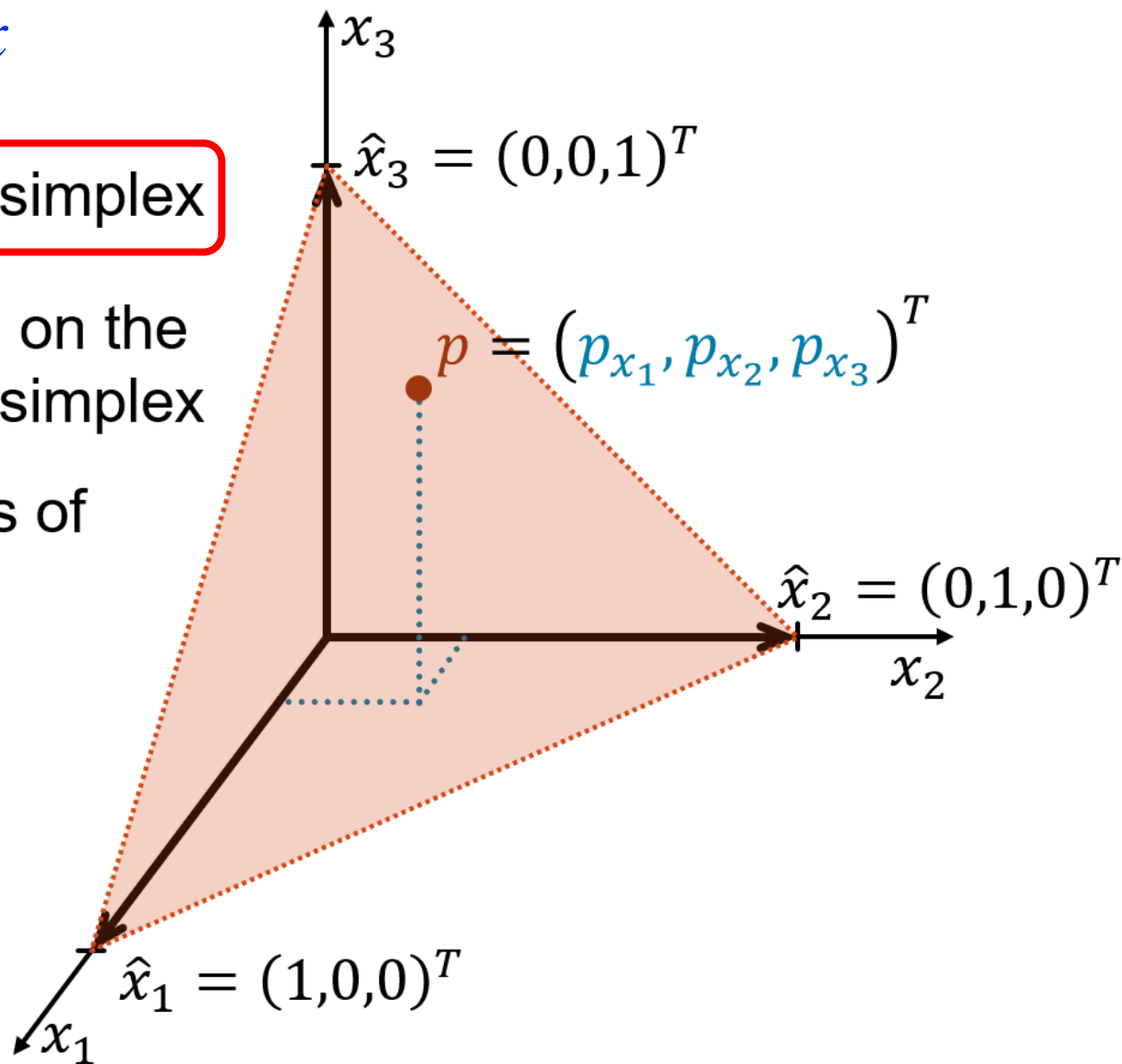


probability simplex

- any point p on the probability simplex

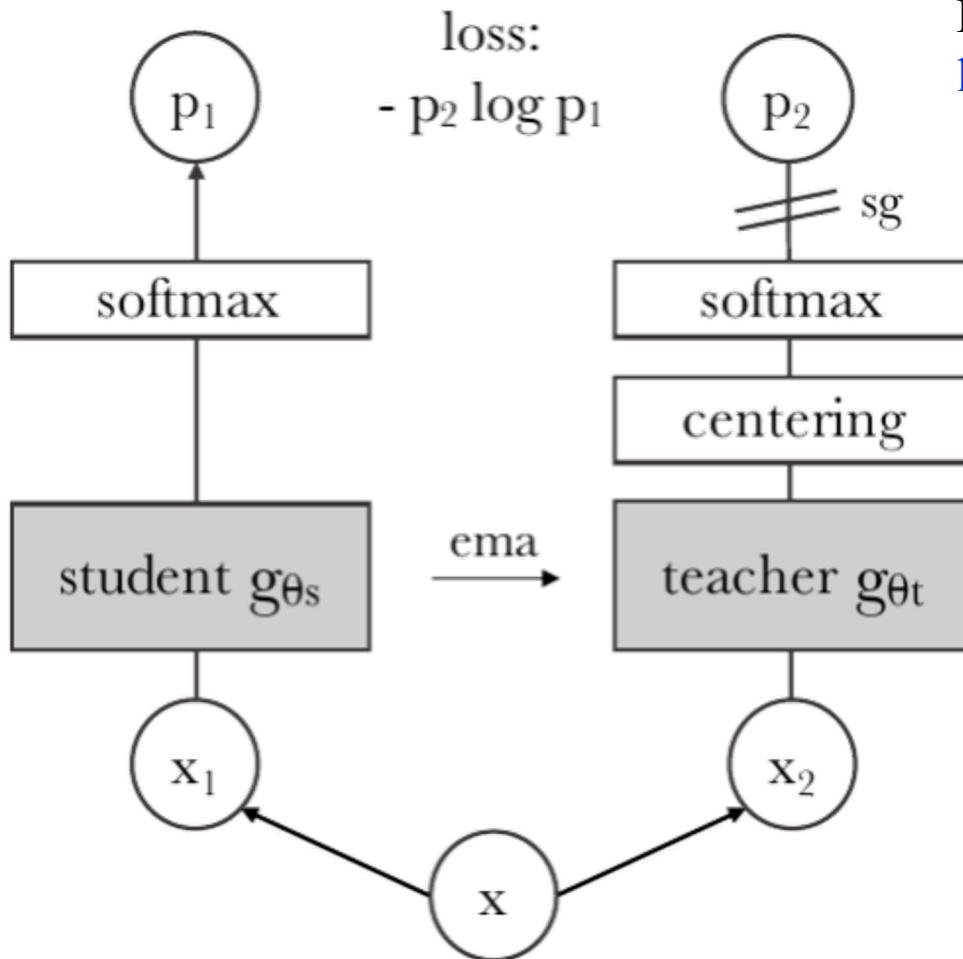
p_{x_i} coordinates of the point p

$$\sum_{i=1}^3 p_{x_i} = 1$$



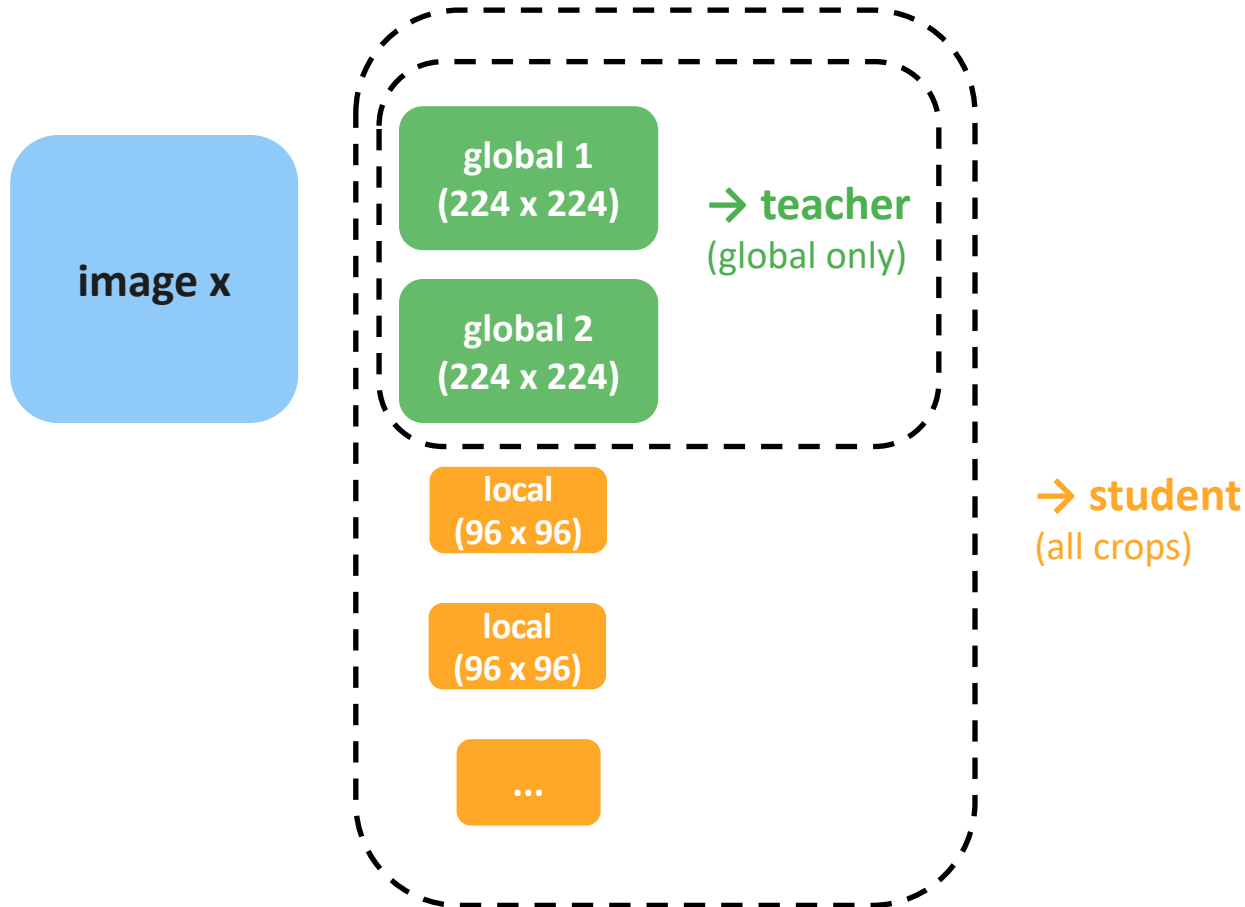
DINO: Self-Distillation Without Labels

Instead of learning a continuous embedding, DINO learns a soft assignment over a set of latent prototypes.



Centering prevents the model from collapsing to a single dominant prototype.

DINO: multi-crop training



The local views do not replace global views, but they force the model to produce representations that are consistent with global semantics even from partial observations.

DINO: why it works especially well with ViTs

1. Token-based structure

ViT processes image as patch token sequence. Attention is spatially interpretable.

2. [CLS] token aggregates global info

[CLS] attends to all patches. In DINO, its attention highlights salient regions.

3. Emergent attention maps

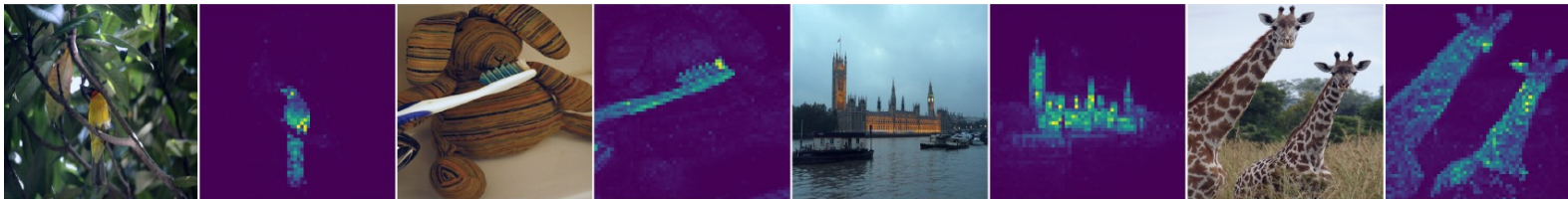
Without any supervision, attention aligns with object boundaries.

This does NOT happen with ResNets or supervised ViTs.

It is a property of self-distillation + ViT + multi-crop together.

4. Practical implication

DINO ViT features transfer well to detection, segmentation, retrieval.



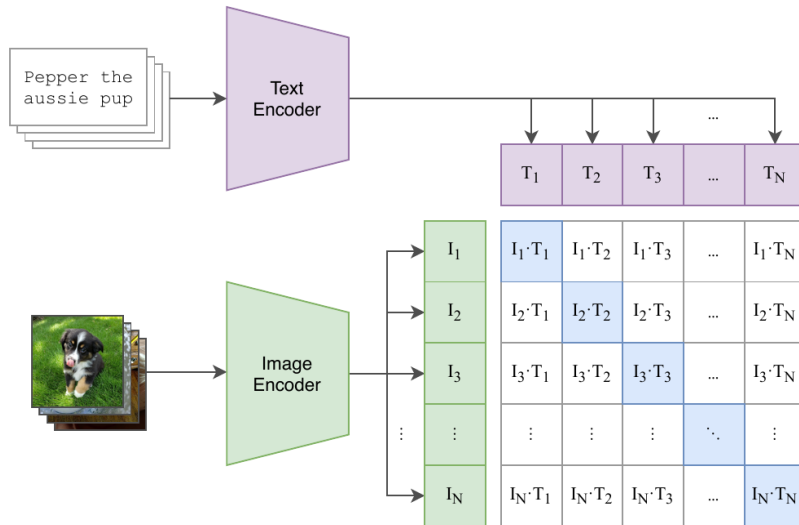
DINO-trained ViTs show emergent object segmentation without supervision.

Multimodal Learning with CLIP

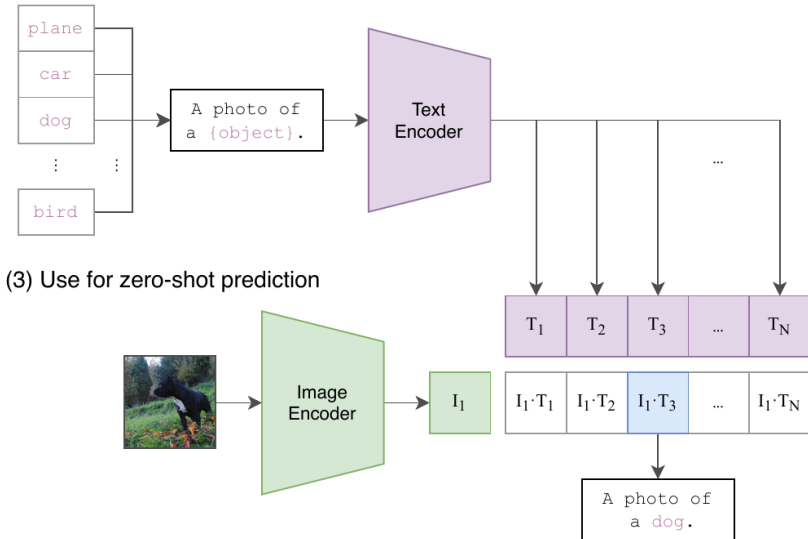
CLIP

Contrastive pre-training on 400M image-text pairs enables zero-shot transfer to any visual task.

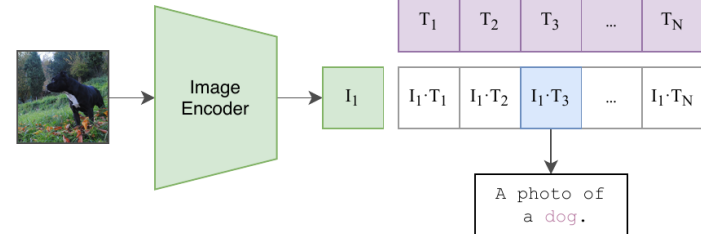
(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



Previous: aug-based supervision

CLIP: (image,text) pair supervision

Zero-shot transfer: describe class in words, classify.

No task-specific training needed.

CLIP

$$L = -\log \frac{\exp\left(\frac{\text{sim}(v_i, t_i)}{\tau}\right)}{\sum_j \exp\left(\frac{\text{sim}(v_i, t_j)}{\tau}\right)}$$

CLIP is essentially SimCLR, but across modalities.

positives = matching image-text

negatives = other samples in batch

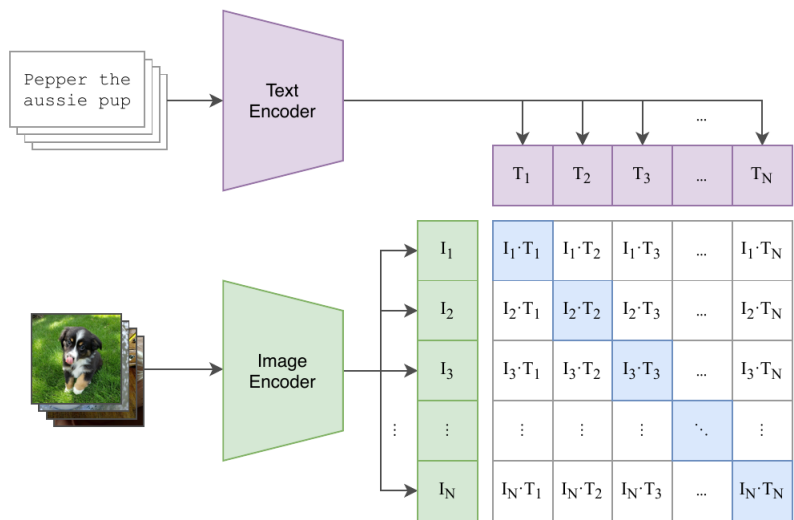
Symmetric Loss

- + image queries text

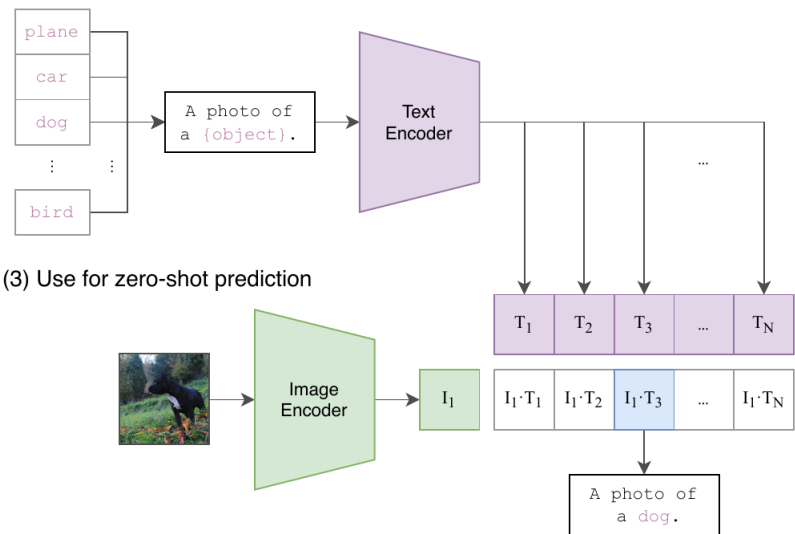
- + text queries image

CLIP: dual-tower architecture

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

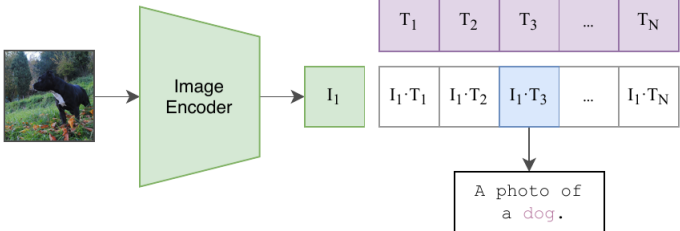


Image encoder:

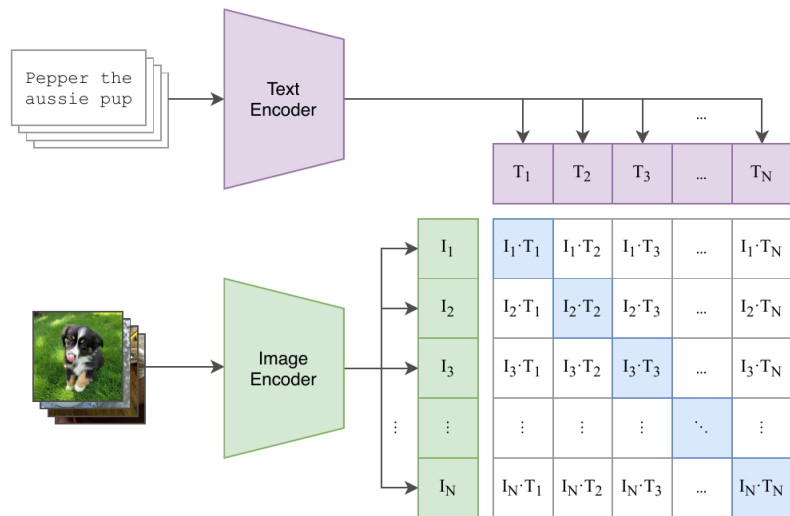
ResNet-50 or ViT (ViT-L/14 is strongest).
Produces image embedding
 $I = \text{image_encoder}(x)$.

Text encoder:

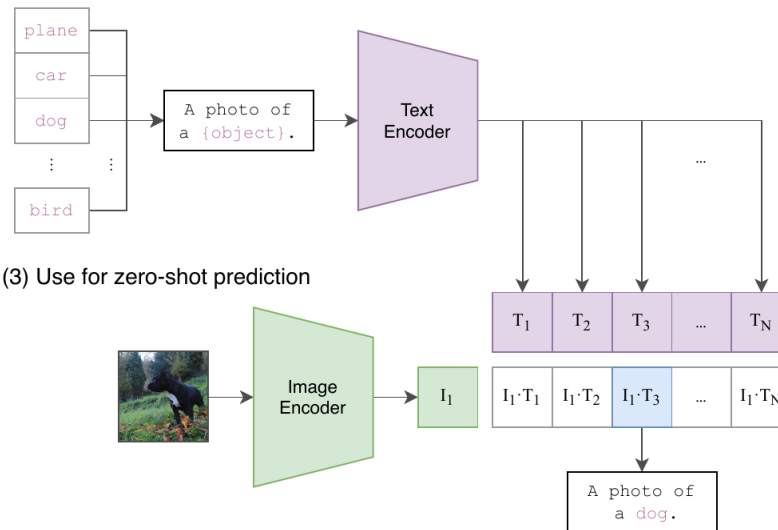
Transformer (63M params).
Produces text embedding
 $T = \text{text_encoder}(\text{caption})$.

CLIP: dual-tower architecture

(1) Contrastive pre-training



(2) Create dataset classifier from label text



Joint embedding space:

Both projected to shared 512-dim space.
L2-normalized. Similarity = cosine.

Training data:

400M (image, text) pairs (WIT dataset).

256 V100 GPUs, batch size 32768.

CLIP: why it matters for the field

1. Scalability

Internet has billions of image-text pairs. No manual labeling needed.

CLIP scales naturally with data, unlike curated datasets.

2. Transfer without fine-tuning

Zero-shot CLIP matches or beats supervised linear probes on many datasets.

Robust to distribution shift (reduces ImageNet robustness gap by 75%).

3. Retrieval and multimodal grounding

Shared embedding space enables image-text retrieval in both directions.

Language grounds visual concepts, visual features ground text descriptions.

4. Foundation model precursor

CLIP is the vision backbone for: DALL-E 2, Stable Diffusion, GPT-4V, LLaVA.

Most works in multimodal models since 2021 are built on or respond to CLIP.

CLIP: limitations

1. Prompt sensitivity

Wording changes evaluation outcomes significantly.

'a photo of a {class}' vs '{class}' can differ by 10+ points.

Prompt engineering is a skill, not a minor detail.

2. Data bias

Internet text-image pairs carry biases of the internet.

Gender, racial, cultural biases in training data become model biases.

CLIP has been shown to exhibit stereotypical associations.

3. Weak fine-grained grounding

CLIP matches global image content with text.

Struggles with: counting ('three dogs'), spatial relations ('left of'),

attribute binding ('the red cube and the blue sphere').

4. Language quality sets the ceiling

Noisy alt-text from the web is often low quality.

The model can only learn what the text supervision provides.

Better text data = better representations (shown by LAION, DataComp).

	AE	MAE	SimCLR	MoCo	BYOL	SimSiam	DINO	CLIP
Supervision	recon	mask	aug	aug	aug	aug	aug	img-txt
Negatives	none	none	batch	queue	none	none	none	batch
Loss	MSE	MSE	NCE	NCE	cos	cos	CE	CE
Anti-collps	btlnk	mask	neg	neg	EMA	sg+pred	cen	neg
Backbone	any	ViT	RN	RN	RN	RN	ViT	both

If SimCLR learns invariances, and DINO learns structure, then CLIP learns semantics.

CLIP bridges perception and language, turning representation learning into a semantic alignment problem.

Final takeaways

1. All methods design a supervision signal.

Reconstruction (AE, MAE), discrimination (SimCLR, MoCo), alignment (BYOL, SimSiam, DINO), language pairing (CLIP).

2. 'What should be the same' is the core design choice.

Augmentation, masking, teacher-student, text pairing. Each defines a different notion of invariance.

3. Know the objective, the target, the anti-collapse mechanism

Also the evaluation protocol for each method.

4. Representation learning = choosing the right invariance.

Too much invariance -> lose useful information.

Too little -> learn shortcuts.

The art is in the balance.

5. Asymmetry prevents collapse in non-contrastive methods.

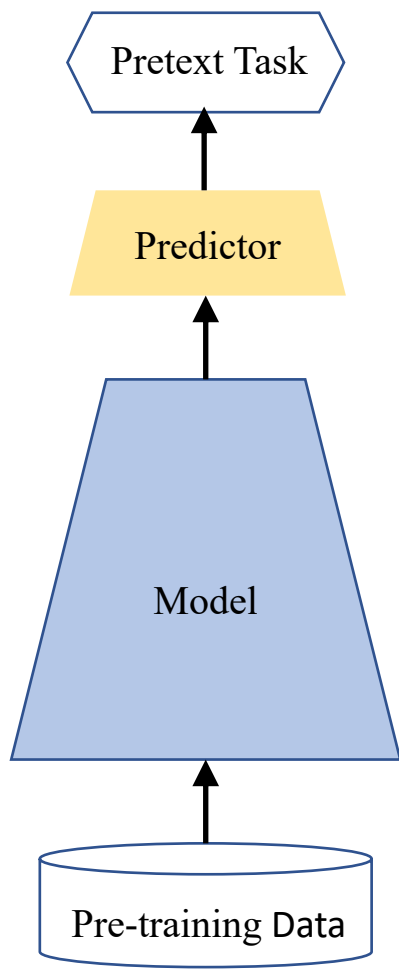
Architecture (predictor)

optimization (stop-grad, EMA),

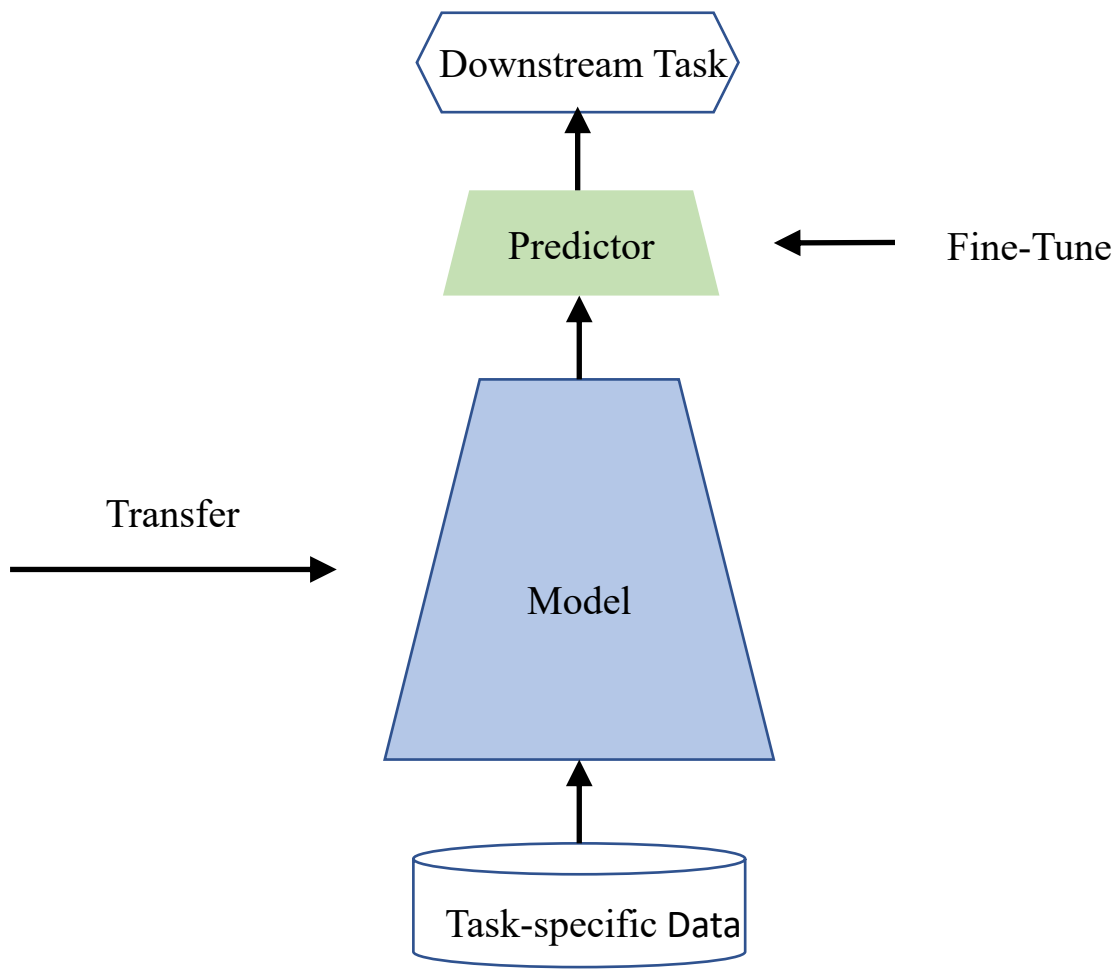
targets (centering, sharpening).

Thank you

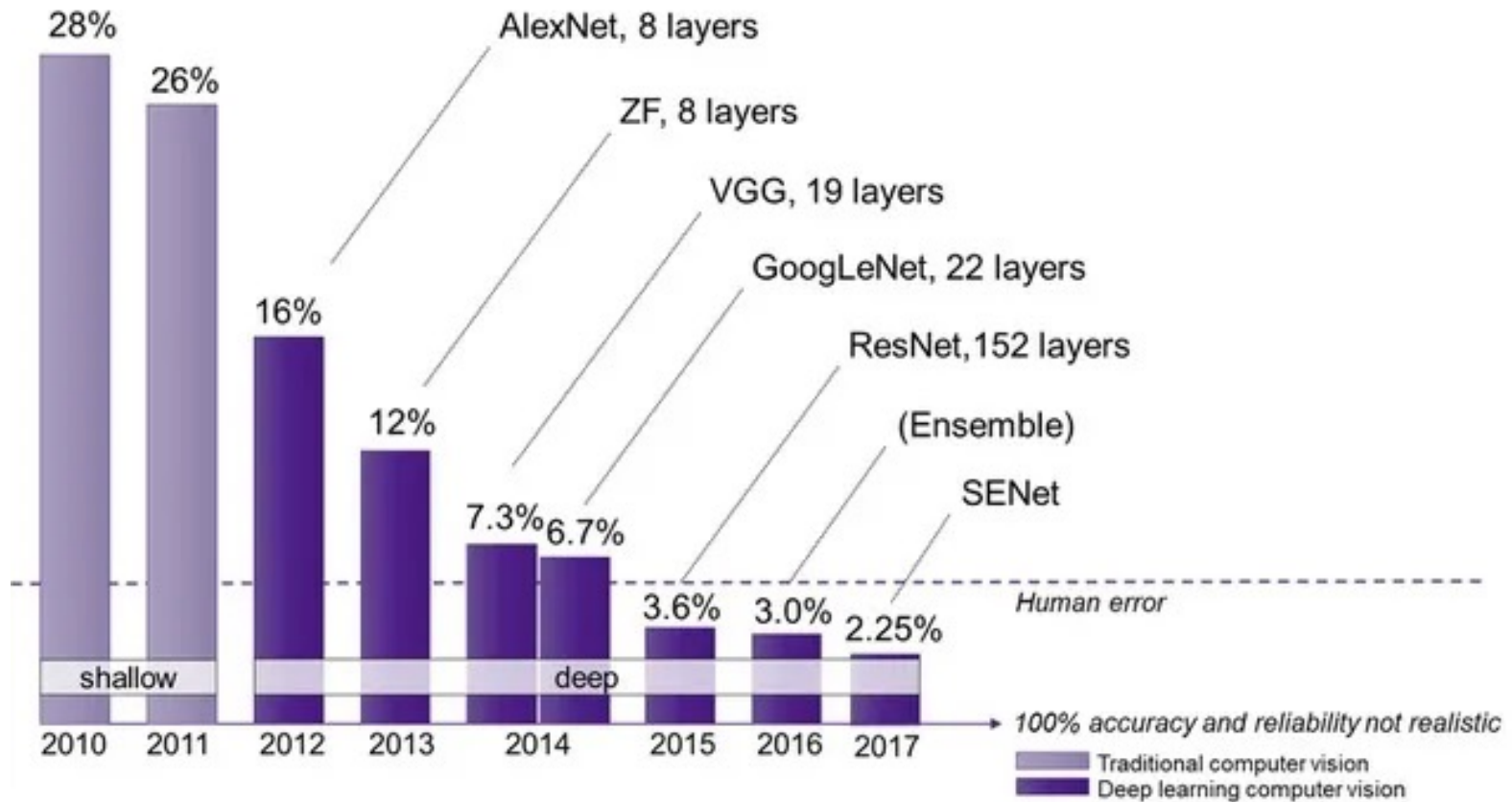
Step 1: Pretrain a model for a pretext task



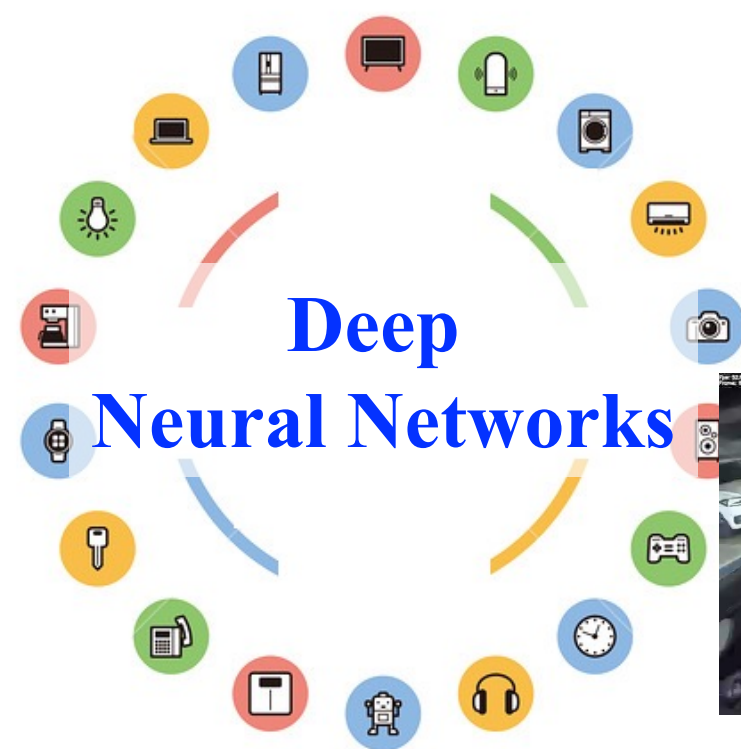
Step 2: Transfer to applications



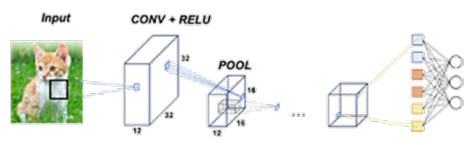
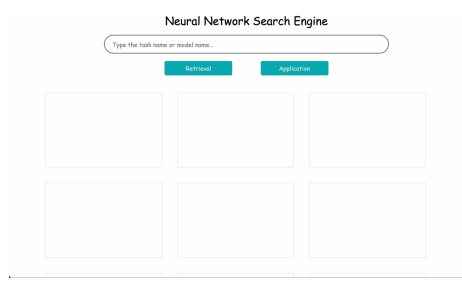
ImageNet Classification Accuracy



Deep Neural Network Design, Search and Optimisation



Neural architecture search
Pruning, compression, quantisation
Evaluating network performance
Computer vision applications, etc.



Our Networks on ImageNet

- 2016 CNNPack [NeurIPS 2016]
75.3% Top 1 ACC
5.82B FLOPs; 97M Params;
- 2017 RedCNN [ICML2017]
74.3% Top 1 ACC
5.0 B FLOPs; 62.29M Params;
- 2018 Versatile Filters [NeurIPS 2018]
74.5% Top 1 ACC
3.0B FLOPs; 41.7M Params
- 2019 Lego Filters [ICML 2019]
74.2% Top 1ACC
2.0B FLOPs; 9.3M Params;
- 2020 GhostNet [CVPR 2020]
74.1% Top 1 ACC
1.2B FLOPs; 6.5M Params;
- 2021 HourNAS [NeurIPS 2021]
74.1% Top 1 ACC
0.29B FLOPs; 5.3M Params;
- 2022 CMT [CVPR 2022]
79.1% Top 1 ACC
0.6B FLOPs; 9.5M Params;
- 2023 TORA [CVPR 2023]
80.3% ACC; 57.5% Robust
- 2024 LocalVMamba
83.7% ACC;

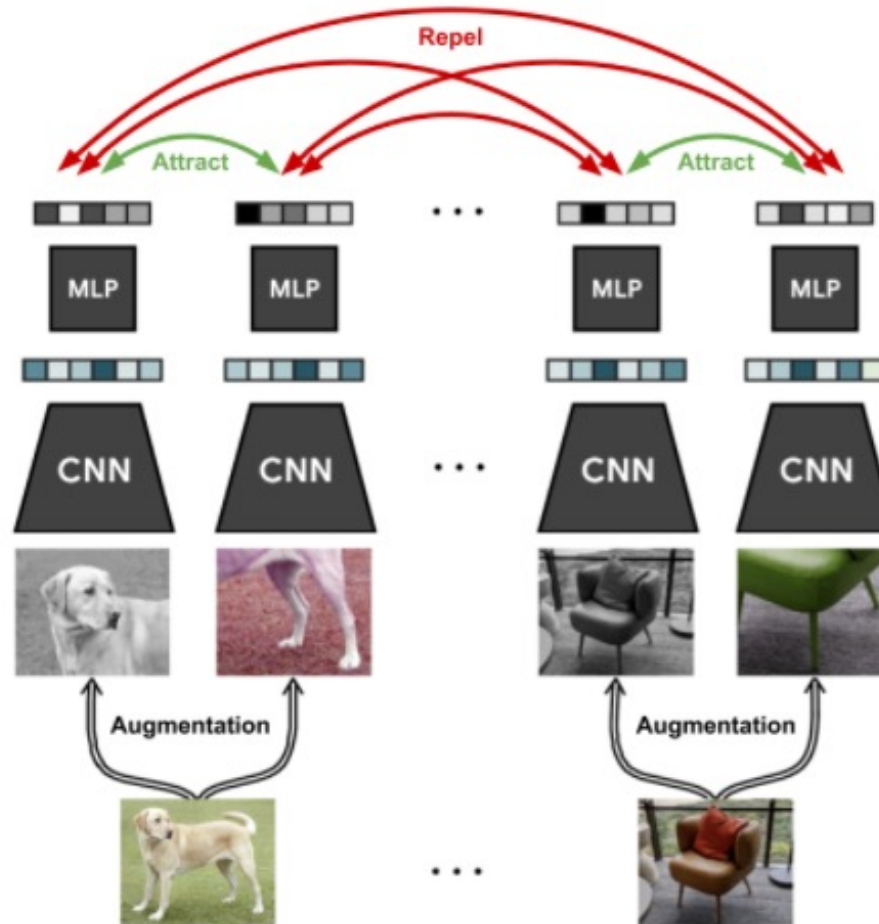
Generative Pre-training

Unsupervised Pre-training

Supervised Pre-training

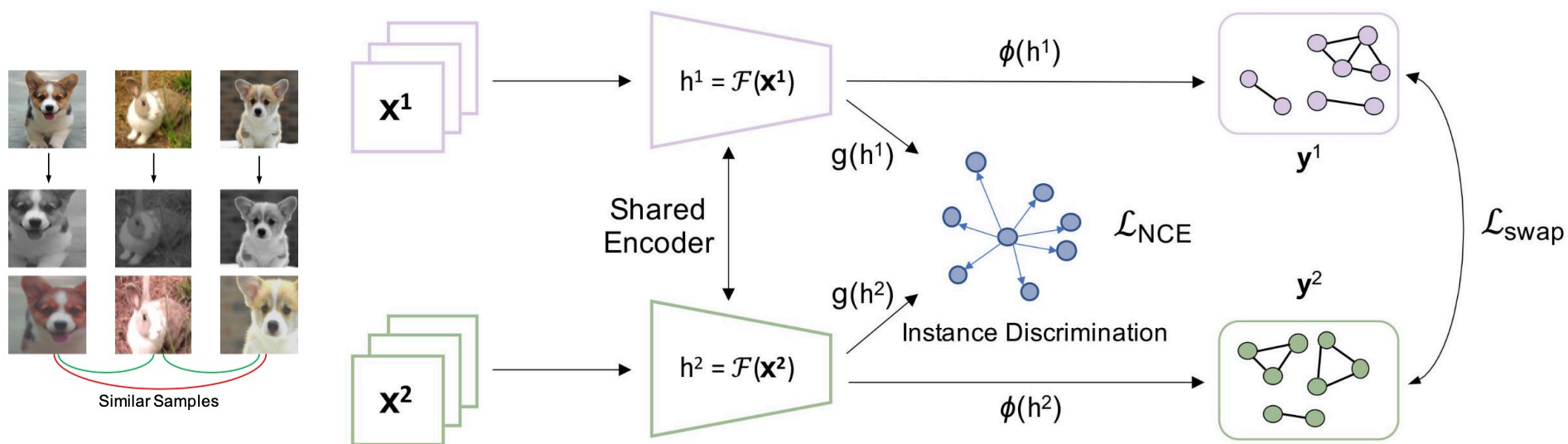
Contrastive Visual Representation Learning

Instance Discrimination is one of the most popular pretext task in contrastive learning, which treats every single instance as a different class. The objective is to *encourage two augmented views of the same image to be pulled closer on the embedding space but pushes apart all the other images away*.



Weakly Supervised Contrastive Learning (WCL)

Traditional contrastive learning treats every image as a separate class, which causes semantic class collision. WCL addresses this by mining nearest-neighbor samples as weak positives, effectively turning instance discrimination into weak semantic discrimination.



Weakly Supervised Contrastive Learning (WCL)

Table 4. Compare to FNCancel on ImageNet-1K

Method	Epochs	GPU(time)	Acc
SimCLR	100	1.00	66.4
FNCancel	100	-	68.1
WCL (Ours)	100	1.01	68.1
SimCLR	1000	10.00	70.3
FNCancel + multi-crops	100	2.85	70.4
WCL (Ours) + multi-crops	100	1.31	71.0

Table 5. Top-1 accuracy under the linear evaluation on ImageNet with the ResNet-50 backbone. The table compares the methods over 200 epochs of pretraining. * denotes multi-crops strategy.

Method	Arch	Param	Epochs	Top-1
Supervised	R50	24	-	76.5
InstDisc [39]	R50	24	200	58.5
LocalAgg [46]	R50	24	200	58.8
SimCLR [6]	R50	24	200	66.8
MoCo [19]	R50	24	200	60.8
MoCo v2 [8]	R50	24	200	67.5
MoCHi [24]	R50	24	200	68.0
CPC v2 [27]	R50	24	200	63.8
PCL v2 [29]	R50	24	200	67.6
SimSiam [9]	R50	24	200	70.0
SwAV [5]	R50	24	200	69.1
SwAV* [5]	R50	24	200	72.7
WCL (Ours)	R50	24	200	70.3
WCL* (Ours)	R50	24	200	73.3

We show the instances that belongs to the same connected components. As we can see, our proposed method can effectively find the semantic similar samples via CCL process



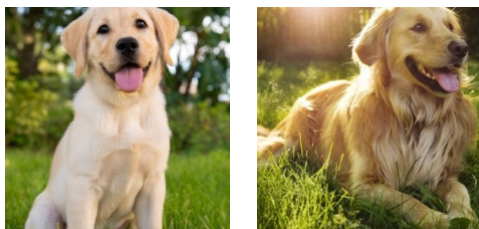
Relational Self-Supervised Learning (ReSSL)

Instance Discrimination

- Augmented views of the same instance should have **identical** feature embeddings.
- Augmented views of different instances should possess **distinct** feature embeddings.



Aggressive Augmented
Positive Pairs



False Negative Pairs
(Class Collision Problem)

Relaxation

- Augmented views of the same instance should have similar, but **not necessarily identical** feature embeddings.
- Augmented views of different instances don't **necessarily need to have widely different** feature embeddings.

Relational Consistency

If we have three instances x_1, x_2, y and z , where x_1, x_2 are two different augmentations of x , y and z are different samples. Then, if x_1 is similar to y but different to z , we wish x_2 can maintain such relationship and vice versa.

Relational Self-Supervised Learning (ReSSL)

TABLE 7

Top-1 accuracy under the linear evaluation on ImageNet with the ResNet-50 backbone. ReSSL with * denote the results without predictor and InfoNCE warm-up.

Method	Backprop	Batch Size	Epochs	Top-1
Supervised	1x	256	120	76.5
<i>1x Backprop Methods</i>				
NPID [78]	1x	256	200	58.5
LocalAgg [90]	1x	128	200	58.8
MoCo V2 [11]	1x	256	200	67.5
MoChi [45]	1x	512	200	68.0
CPC v2 [50]	1x	512	200	63.8
PCL v2 [54]	1x	256	200	67.6
AdCo [42]	1x	256	200	68.6
InfoMin [69]	1x	256	200	70.1
ISD [67]	1x	256	200	69.8
ReSSL * (1 Crop) [89]	1x	256	200	69.9
ReSSL (1 Crop)	1x	1024	200	72.0
<i>2x Backprop Methods</i>				
CLSA-Single [75]	2x	256	200	69.4
SimCLR [9]	2x	4096	200	66.8
SimSiam [12]	2x	256	200	70.0
MoCo V2 - Symmetric [12]	2x	256	200	69.9
WCL [88]	2x	4096	200	70.3
NNCLR [26]	2x	256	200	70.7
BYOL [35]	2x	4096	300	72.5
Barlow Twins [84]	2x	2048	300	71.4
MoCo V3 [13]	2x	4096	300	72.8
SwAV [7]	2x	4096	400	70.1
W-MSE 4 [28]	2x	1024	400	72.6
ReSSL * (2 Crop) [89]	2x	256	200	71.4
ReSSL (2 Crops)	2x	1024	200	73.2

TABLE 1
Effect of different τ_t and τ_s for ReSSL

Dataset	τ_s	$\tau_t = 0.01$	$\tau_t = 0.02$	$\tau_t = 0.03$	$\tau_t = 0.04$	$\tau_t = 0.05$	$\tau_t = 0.06$	$\tau_t = 0.07$	$\tau_t = 0.08$	$\tau_t = 0.09$	$\tau_t = 0.10$
CIFAR-10	0.1	89.72	90.16	90.62	90.76	90.46	90.21	89.95	89.67	89.61	89.25
CIFAR-10	0.2	90.18	90.42	90.42	89.91	89.97	90.00	89.84	89.19	88.88	88.63
CIFAR-100	0.1	63.63	64.61	65.03	64.60	64.26	63.19	62.54	60.56	60.17	58.18
CIFAR-100	0.2	63.12	63.09	62.08	61.93	61.20	60.48	59.25	58.52	58.11	56.36
STL-10	0.1	87.82	88.54	88.89	88.41	87.60	87.24	86.87	84.04	83.67	83.59
STL-10	0.2	87.66	87.28	86.51	86.19	85.75	85.75	85.58	85.61	85.37	84.52
Tiny ImageNet	0.1	45.66	45.62	46.64	46.06	45.36	41.22	37.02	36.03	32.54	32.28
Tiny ImageNet	0.2	45.64	45.74	43.72	42.62	42.02	40.72	38.24	34.74	34.12	30.10

TABLE 4
Effect of different augmentation for teacher model

Random Resized Crops	Random Flip	Color Jitter	GrayScale	Gaussian Blur	CIFAR-10	CIFAR-100	STL-10	Tiny ImageNet
					79.33	52.77	86.01	36.76
✓					90.62	64.98	88.62	46.38
	✓				80.38	51.60	86.24	38.20
		✓			78.47	51.73	85.92	36.64
			✓		78.44	49.76	87.12	36.32
				✓	78.54	50.65	85.34	36.20
✓					90.76	65.03	88.89	46.64
✓			✓		89.80	62.87	87.45	43.98
✓				✓	89.56	62.61	88.65	42.52
✓					89.18	62.75	87.34	46.18
✓	✓		✓		90.15	63.55	87.21	44.00
✓		✓		✓	89.63	62.95	88.47	41.88
✓	✓			✓	89.43	62.37	87.22	45.30
✓		✓	✓	✓	86.88	58.60	85.05	38.66

Generative Pre-training

Unsupervised Pre-training

Supervised Pre-training

Image Pixels Generation



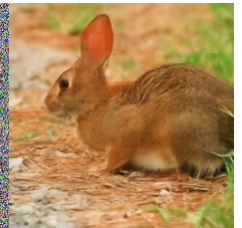
Original

Masked ($\gamma = 75\%$) Reconstructed



Noisy ($\sigma = 70$)

Reconstructed



Noisy ($\sigma = 140$)

Reconstructed

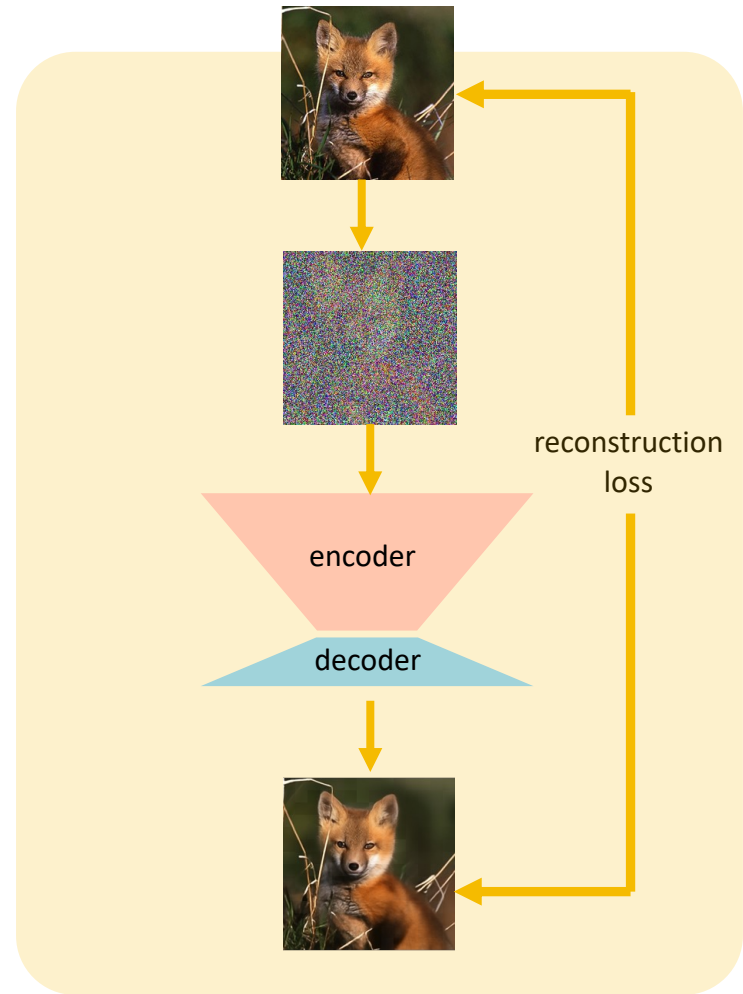
Noisy Image Modeling

- Use denoising as the pretext task
- Learn noise-invariant features

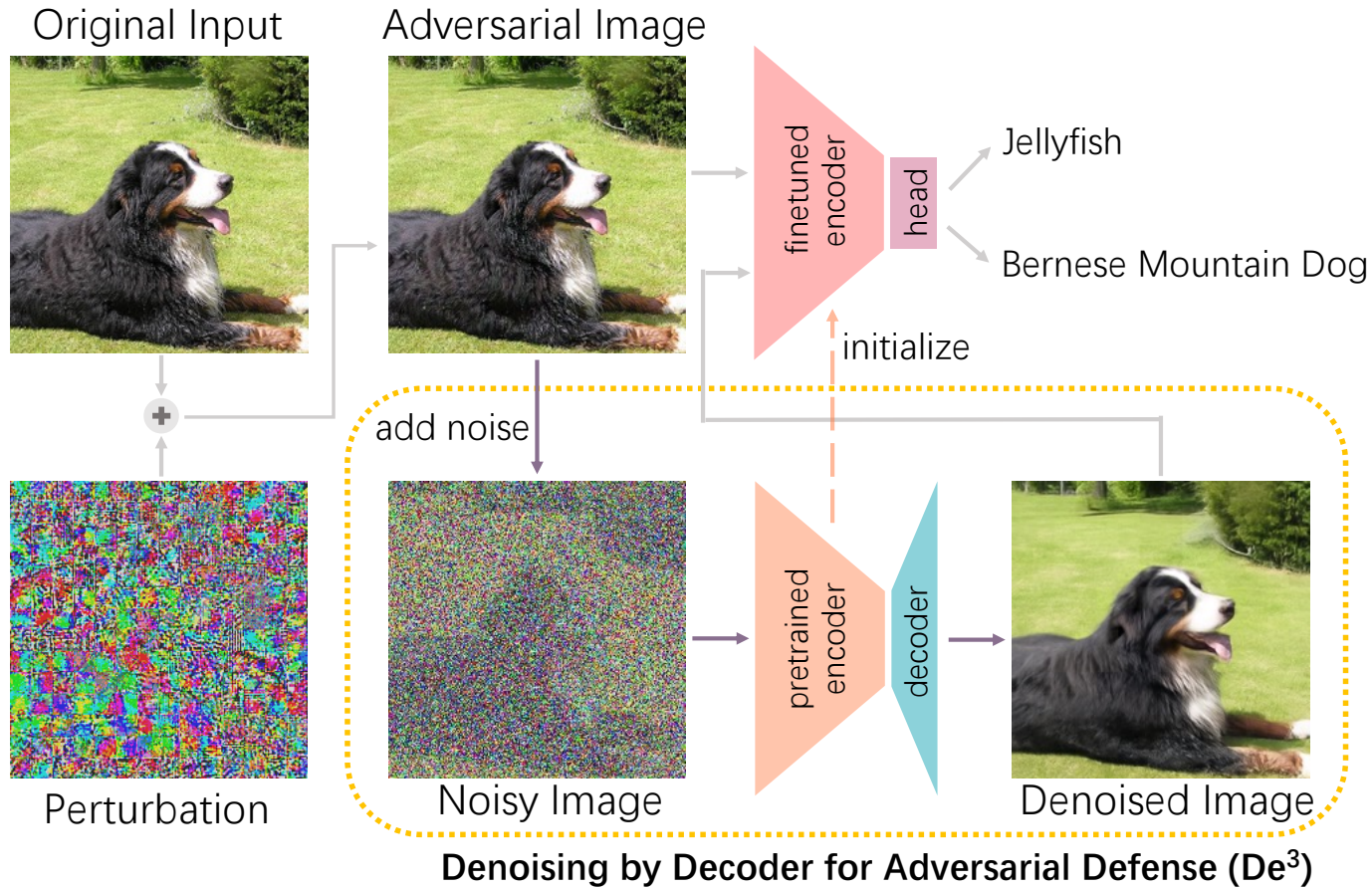
$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \Omega} \mathcal{D}(g_{\phi}(f_{\theta}(\hat{\mathbf{x}}_{NIM})), \mathbf{x})$$

↓

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \Omega} \mathcal{D}'_{\phi, \phi'}(f_{\theta}(\hat{\mathbf{x}}_{NIM}), f_{\theta}(\mathbf{x}))$$



Denoising by Decoder for Adversarial Defense



Experiment Results

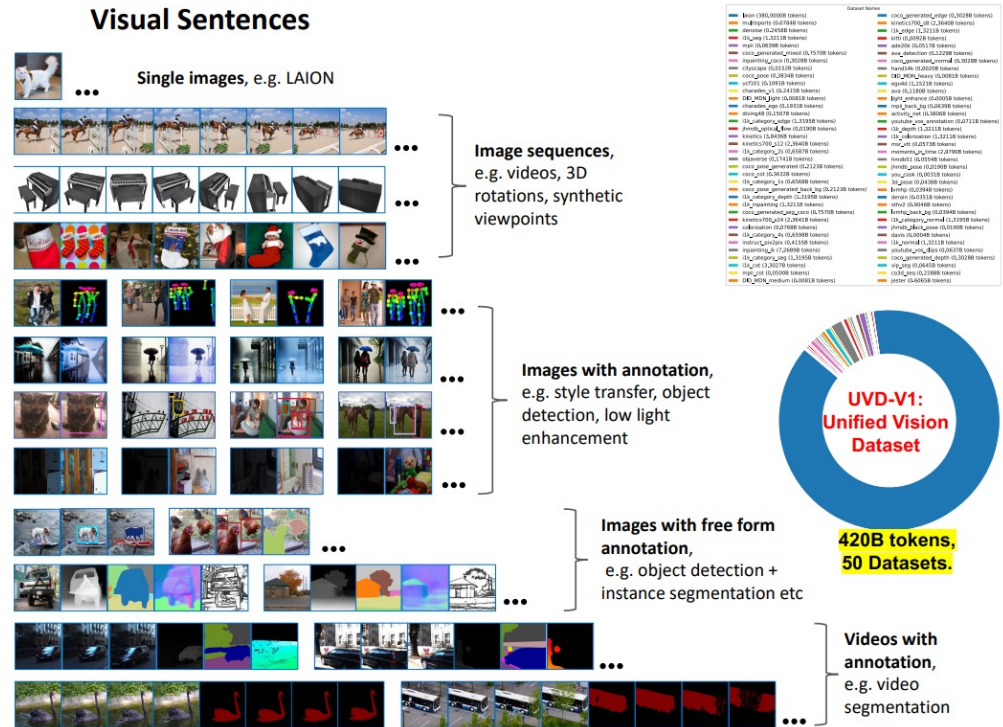
Table 1: Comparisons of ImageNet-1K clean and robustness accuracies (%) between MIM and NIM-MIM, with or without the proposed De³ defense. [†]De³ refers to the defense method that adapts De³ to MIM methods (i.e., *de-masking* by decoder for adversarial defense). Numbers following De³ indicate the degradation level used in defense, where γ is the mask ratio and σ is the noise scale. NIM-MAE models that use De³ for defense are finetuned on denoised images from the pretrained model.

Model	MAE [17]		NIM-MAE			SimMIM [38]		NIM-SimMIM		
De ³	None	$\gamma=0.75$	None	$\sigma=70$	$\sigma=140$	None	$\gamma=0.6$	None	$\sigma=70$	$\sigma=140$
Clean	83.05	44.96	82.58	78.68	70.69	83.62	43.05	82.76	76.23	68.76
FGSM	28.29	36.38	31.66	53.66	53.12	30.38	34.29	31.90	51.84	51.67
PGD-10	0.25	11.58	0.31	34.61	39.82	1.13	7.39	2.02	33.41	37.45
AA	0.00	2.58	0.00	23.24	33.70	0.00	1.82	0.00	21.70	32.46

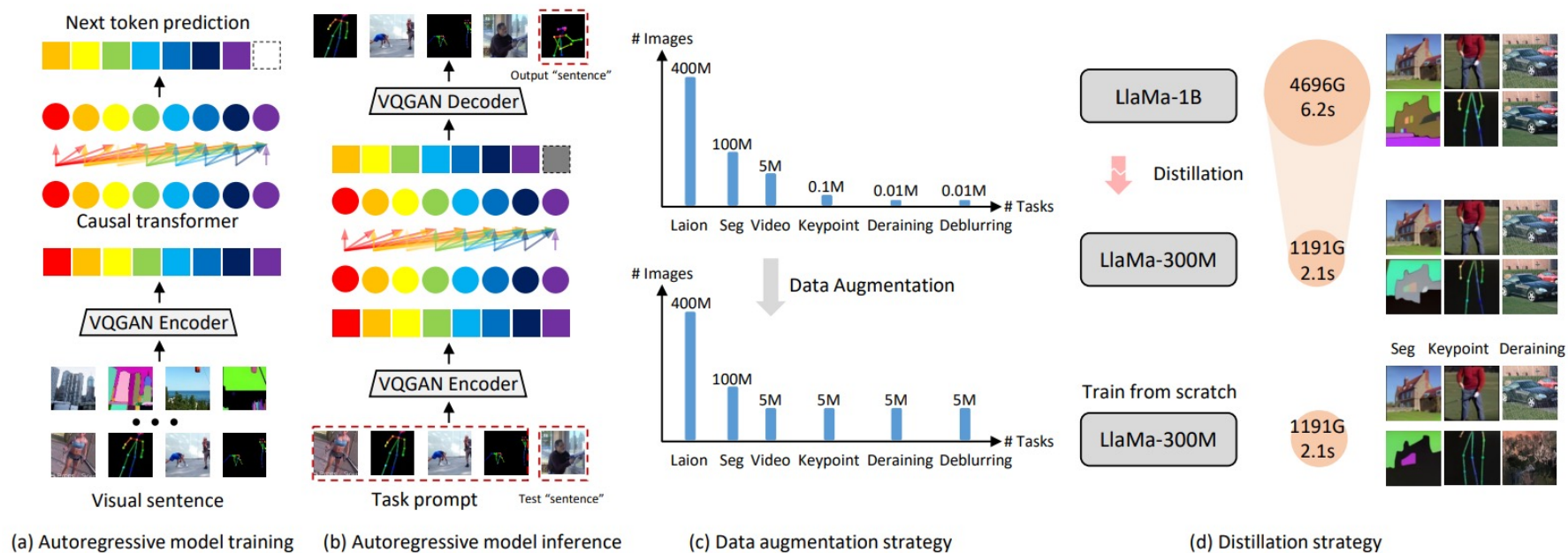
Further scaling up...

Background: large vision model

- ❑ Data diversity on various standard vision tasks.
- ❑ Increase model size and data size.
- ❑ Many different vision tasks can now be “solved” by a single vision model.



Motivation & pipeline of DeLVM



Data-efficient Large Vision Models through Sequential Autoregression, ICML 2024

Dataset & Model

Dataset:

SA-1B, Rain13K, COCO-pose, HDvila-100m, LAION.

SA-1B (35.05%; 28.97 billion tokens). SA-1B is a large-scale multimodal dataset designed for training general-purpose object **segmentation** models. It comprises 1B high-resolution images, along with corresponding high-quality segmentation masks.

Rain13K (16.96%; 14.02 billion tokens). Rain13K serves as the most commonly utilized training dataset for **rain removal**. The original dataset comprises 13,712 clean-rain image pairs.

COCO-Pose (12.92%; 10.68 billion tokens). A dataset for **human pose detection**. The original version is a subset of COCO dataset and has 250K images, with annotations for 17 human keypoints, such as eyes, hands, legs, feet, etc.

Detailed configurations of our used models.

Model	Hidden dim.	MLP dim.	#heads	#layers
LLaMA-1B	2048	5504	16	22
LLaMA-300M	1024	2688	8	22
LLaMA-80M	768	3072	12	12

HDvila-100m (1.25%; 1.03 billion tokens). A large-scale **video-language** multimodal dataset containing 100 million high-resolution, diverse, and video clips and 100 million automatically generated text descriptions. We exclusively utilized a subset of the video data to endow our model with the capability for continuous inference.

LAION-400M (33.83%; 27.96 billion tokens). A large-scale multimodal dataset containing 400 million English **image-text** pairs. It covers a wide range of topics and includes both text descriptions and corresponding images.

Part-1: Data-efficient Training: Data Augmentation Strategy

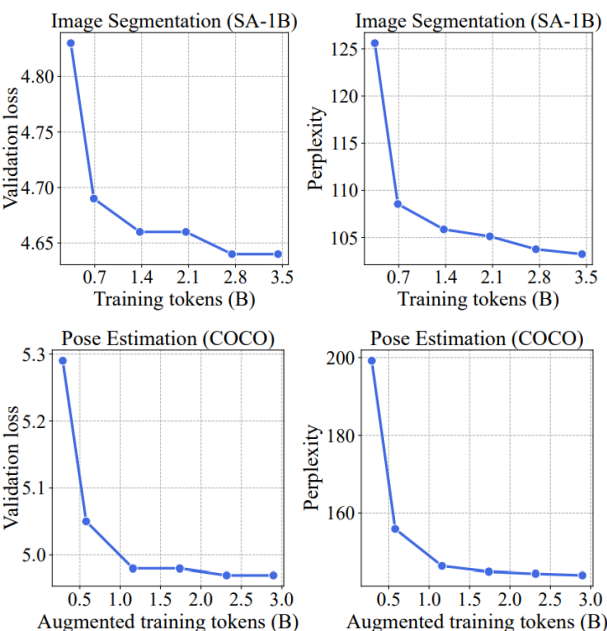
Data augmentation vs. Collecting new data (LLaMA-300M [2])

Subsets of SA-1B (12K training steps)

- Tokens: 34 billion
- In segmentation task with ample available data, enhanced model performance is observed upon introducing more training data.

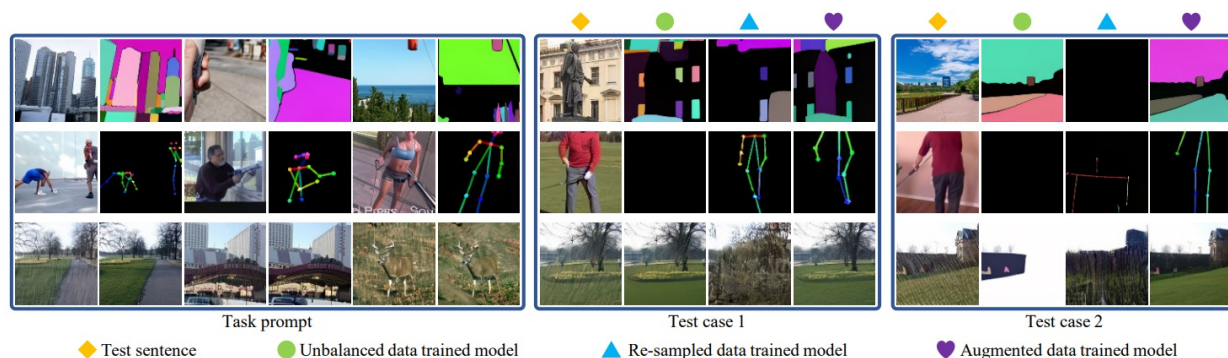
Augmented COCO-Pose (12K training steps)

- Tokens: 0.03 billion
- In human pose estimation task with limited data, augmenting the dataset has a comparable impact to the introduction of new training data.



Dataset augmentation yielded the best overall performance. "-" indicates collapsed results.

Dataset configuration	Image Segmentation (10% of SA-1B: 3.434B tokens)			Pose Estimation (COCO-Pose: 0.029B tokens)			Image Deraining (Rain13K: 0.007B tokens)		
	loss ↓	accuracy ↑	perplexity ↓	loss ↓	accuracy ↑	perplexity ↓	loss ↓	accuracy ↑	perplexity ↓
Unbalanced	4.48	20.32	88.59	4.95	19.86	141.91	5.69	11.89	279.88
Balanced by re-sampling	5.01	16.68	151.66	16.11	15.75	-	21.73	4.53	-
Balanced by augmentation	4.68	18.79	107.77	4.94	20.24	140.39	5.64	11.57	269.04

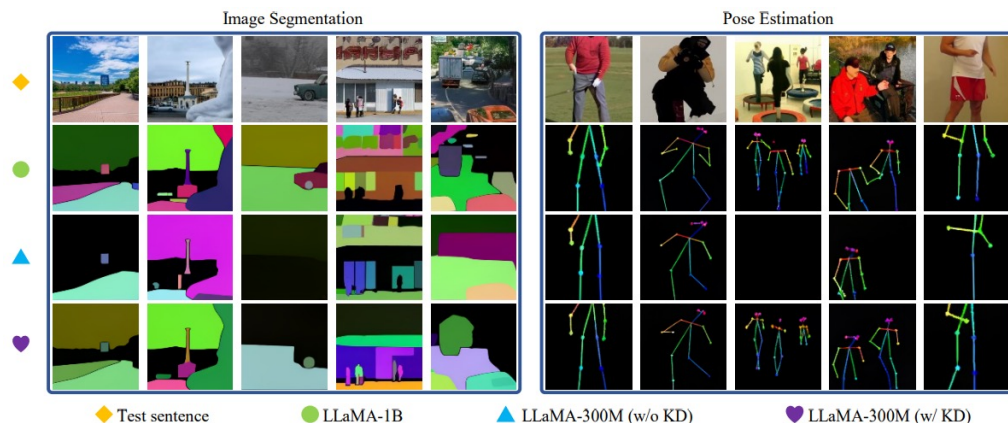


Part-2: Low Power Consumption and Tiny Model: KD

The introduction of KD proves beneficial in enhancing the single-task performance of LLMs.

Model	KD	loss ↓	accuracy ↑	perplexity ↓
<i>Image Segmentation (10% of SA-1B)</i>				
LLaMA-1B	-	4.50	20.18	90.04
LLaMA-300M	✗	4.64	19.17	103.24
LLaMA-300M	✓	4.59	19.48	98.72

Model	KD	loss ↓	accuracy ↑	perplexity ↓
<i>Pose Estimation (COCO-Pose)</i>				
LLaMA-1B	-	4.90	20.96	134.07
LLaMA-300M	✗	4.97	20.46	144.08
LLaMA-300M	✓	4.91	20.80	135.91



The introduction of KD proves beneficial in enhancing the multi-task performance of LLMs.

Model	KD	Image Segmentation			Pose Estimation			Image Deraining		
		loss ↓	accuracy ↑	perplexity ↓	loss ↓	accuracy ↑	perplexity ↓	loss ↓	accuracy ↑	perplexity ↓
LLaMA-1B	-	4.55	19.72	94.75	4.86	20.77	129.95	5.53	12.20	245.33
LLaMA-300M	✗	4.68	18.79	107.76	4.94	20.24	140.39	5.63	11.57	271.38
LLaMA-300M	✓	4.67	18.84	106.81	4.93	20.32	139.27	5.62	11.93	269.04

LLaMA-300M w/o KD vs. LLaMA-300M w/ KD on LSP (Leeds Sports Pose) dataset.



LLaMA-300M w/o KD vs. LLaMA-300M w/ KD on OCHuman (Occluded Human) dataset.

